



Tunisian Republic
Ministry of Higher Education and Scientific Research
University of Sousse
**Higher Institute of Computer Science
and Communication Technologies
(ISITCOM)**



End of Studies Project Report

Submitted in partial fulfillment of the requirements for the
National Engineering Degree — Specialty: Teleinformatics

Presented by:

Asma Daoussi

Development of a Digital Asset Marketplace

for Inherited Games Studio (IGS)

Carried out at: Inherited Games Studio (IGS), Tunisia

Academic Supervisor: Mr. Sami Ben Amor

Company Supervisor: Mme. Nour Ltaief

Defended on /06/2026 **before the jury composed of:**

Jury President: Mr. _____

Rapporteur: Mr. _____

Academic Year: 2025–2026

Authorization for Defense

Company Supervisor

I hereby authorize the student to submit their End of Studies Project Report for the jury defense.

Madame Nour LTAIEF

Company Supervisor, Inherited Games Studio

Signature: _____

Academic Supervisor

I hereby authorize the student to submit their End of Studies Project Report for the jury defense.

Monsieur Sami BEN AMOR

Academic Supervisor, ISITCOM — University of Sousse

Signature: _____

Dedication

To my beloved family,

*who have been my unwavering pillar of strength
throughout every step of this journey.*

To my parents,

*whose sacrifices, endless encouragement, and unconditional love
have been the foundation upon which I built every dream.*

To my brothers and sisters,

*for the laughter, the support, and the belief
that always pushed me forward.*

To my friends and colleagues,

*who made these long study years memorable
and filled them with moments of joy and solidarity.*

*To every person who believed in me
even when I doubted myself —*

this work is for you.

Asma Daoussi

Acknowledgments

First and foremost, I would like to express my sincere gratitude to **Mr. Sami Ben Amor**, my academic supervisor, for his continuous guidance, insightful advice, and unwavering support throughout the realization of this project. His expertise and constructive feedback have been invaluable in shaping both the technical direction and the quality of this work.

I extend my heartfelt thanks to **Mme. Nour Ltaief**, my company supervisor at Inherited Games Studio, for welcoming me into the team, sharing her professional experience, and providing the resources and environment necessary to bring this project to life. Her trust in my capabilities and her constant encouragement made this internship an enriching and rewarding experience.

I would like to express my appreciation to all the staff at **Inherited Games Studio (IGS)** for their warm welcome and their readiness to assist me whenever needed. Working alongside such a talented and passionate team has been a privilege.

My gratitude also goes to the faculty and staff of **ISITCOM — Higher Institute of Computer Science and Communication Technologies** at the **University of Sousse**, whose dedication to quality education provided me with the theoretical and practical foundations that made this project possible.

I am also deeply thankful to the members of the **jury** for dedicating their time to evaluate this work, and for the honor they grant me by participating in its defense.

Finally, I would like to thank my family and friends for their patience, encouragement, and moral support during the long nights and challenging moments of this project.

Asma Daoussi
Sousse, 2026

Contents

Dedication	i
Acknowledgments	ii
List of Figures	ix
List of Tables	xi
List of Acronyms	xii
General Introduction	1
1 General Context	4
1.1 Host Organization: Inherited Games Studio	4
1.1.1 Company Profile	4
1.1.2 Organizational Structure	5
1.2 Project Context and Problem Statement	5
1.2.1 Context	5
1.2.2 Problem Statement	6
1.3 Comparative Study of Existing Solutions	6
1.3.1 The Global 3D Marketplace Landscape	6
1.3.2 Comparative Summary Table	8
1.3.3 Conclusion of the Comparative Study	9
1.4 Proposed Solution Overview	9
1.5 Development Methodology: Scrum / Agile	10
1.5.1 Comparative Study of Software Development Methodologies . .	10
1.5.2 Why Scrum?	13
1.5.3 The Scrum Framework	13
1.5.4 Sprint Planning Overview	14
1.6 Development Environment and Technology Stack	15
1.6.1 Technology Stack Overview	16
1.6.2 Technology Badges	17
1.6.3 Key Technology Choices	17

2	Requirements Analysis	19
2.1	Actors Identification	19
2.1.1	Actor Descriptions	19
2.2	Functional Requirements	20
2.2.1	Authentication & User Management	20
2.2.2	Asset Catalog	20
2.2.3	Asset Upload & Management (Seller)	21
2.2.4	Shopping Cart & Orders	21
2.2.5	Reviews & Ratings	21
2.2.6	Admin Panel	21
2.2.7	Notifications	22
2.2.8	Subscriptions	22
2.2.9	Intern Role Features	22
2.2.10	Community Features	22
2.2.11	IGS Workspace	22
2.3	Non-Functional Requirements	23
2.4	Global Use Case Diagram	24
2.5	System Architecture	24
2.5.1	Overall Architecture	24
2.5.2	Technology Stack	25
2.5.3	Backend Layered Architecture	25
3	Sprint 1: Platform Foundation	27
3.1	Sprint Backlog	27
3.2	Analysis and Design	28
3.2.1	Database Schema — Core Tables	28
3.2.2	Authentication Flow	30
3.2.3	RBAC Architecture	31
3.3	Implementation	31
3.3.1	Backend Infrastructure	31
3.3.2	Authentication Middleware	32
3.3.3	RBAC Permission Middleware	32
3.3.4	Frontend Authentication Context	32
3.4	Testing	33
3.5	Sprint Review	33
3.5.1	Deliverables	33
3.5.2	Sprint Screenshots	34

4	Sprint 2: Marketplace Core Features	35
4.1	Sprint Backlog	35
4.2	Analysis and Design	36
4.2.1	Database Schema — Asset and Commerce Tables	36
4.2.2	Asset Upload Workflow	38
4.2.3	Payment Flow — Stripe	39
4.2.4	Asset Viewer System	40
4.3	Implementation	40
4.3.1	Asset Catalog with Filtering	40
4.3.2	Interactive 3D Model Viewer	40
4.3.3	Persistent Shopping Cart	40
4.3.4	File Storage via Supabase	41
4.4	Testing	41
4.5	Sprint Review	42
4.5.1	Deliverables	42
4.5.2	Sprint Screenshots	42
5	Sprint 3: Content Management and Administration	44
5.1	Sprint Backlog	44
5.2	Analysis and Design	45
5.2.1	Database Schema — Reviews, Notifications, Subscriptions	45
5.2.2	Admin Moderation Use Case	47
5.2.3	Subscription Plans	47
5.3	Implementation	47
5.3.1	Review and Rating System	47
5.3.2	Notification System	48
5.3.3	Admin Moderation Panel	48
5.3.4	Platform Analytics Dashboard	48
5.4	Testing	49
5.5	Sprint Review	49
5.5.1	Deliverables	49
5.5.2	Sprint Screenshots	50
6	Sprint 4: Role-Based Workflows and Community Features	52
6.1	Sprint Backlog	52
6.2	Analysis and Design	53
6.2.1	Database Schema — Intern and Advanced Features	53
6.2.2	Intern Asset Request Workflow	55
6.2.3	Peer-to-Peer Trading System	55
6.2.4	Seller Analytics	56

6.3	Implementation	56
6.3.1	Intern Asset Request Controller	56
6.3.2	Seller Analytics	56
6.3.3	Asset Edit Requests	57
6.3.4	Community Groups	57
6.4	Testing	58
6.5	Sprint Review	58
6.5.1	Deliverables	58
6.5.2	Sprint Screenshots	59
7	Sprint 5: Internal Collaboration Platform	60
7.1	Sprint Backlog	60
7.2	Analysis and Design	61
7.2.1	IGS Workspace Architecture	61
7.2.2	Database Schema — Workspace and Chat	62
7.2.3	Role-Based Navigation	63
7.2.4	Real-Time Chat Architecture (Socket.io)	64
7.3	Implementation	64
7.3.1	Workspace Layout and Auth Guard	64
7.3.2	Real-Time Chat System	65
7.3.3	Task Management	65
7.3.4	Workspace Dashboard	65
7.4	Testing	66
7.5	Sprint Review	66
7.5.1	Deliverables	66
7.5.2	Sprint Screenshots	67
8	Sprint 6: Artificial Intelligence Integration	69
8.1	Sprint Backlog	69
8.2	AI System Architecture	70
8.2.1	Design Principles	70
8.2.2	AI Model Specification	71
8.3	AI Review Analysis (Completed)	72
8.3.1	Feature Description	72
8.3.2	Analysis Output Structure	72
8.3.3	Automated Negative Review Flagging	72
8.4	Marketplace Assistant Chatbot (In Progress)	73
8.4.1	Design	73
8.4.2	Tool Use (Function Calling)	73
8.4.3	Frontend Widget	74

8.4.4	Remaining Work	74
8.5	Future AI Perspectives	74
9	Sprint 7: AI-Enhanced Services and Catalog Expansion	76
9.1	Sprint Backlog	76
9.2	AI Recommendation Engine	77
9.2.1	Architecture — Two-Stage Pipeline	77
9.2.2	User Signal Tracking	78
9.2.3	Content-Based Scoring	78
9.2.4	AI Reranking	79
9.2.5	Cold Start Handling	79
9.3	IGS Marketplace Bot	79
9.3.1	Overview	79
9.3.2	AI Model Configuration	80
9.3.3	Context Injection	80
9.3.4	Frontend Widget Features	80
9.4	Upload Form Improvements	81
9.5	Mandatory Format Selection Before Download	81
9.6	Catalog Expansion — New Asset Categories	82
9.7	Seed Data — Real Assets on Supabase CDN	83
9.8	Database Changes	84
	General Conclusion and Perspectives	85
	Bibliography	88
	Abstract	90

List of Figures

1	Global 3D model marketplace market growth forecast — CAGR 11.2% (Source: P Market Research, 2024)	1
1.1	Global Architecture of the IGS Platform	10
1.2	Sprint timeline overview — 7 sprints over 20 weeks	15
1.3	Technology stack used across frontend, backend, data layer, and tooling	17
2.1	Global Use Case Diagram	24
3.1	Sprint 1 — Core Entity-Relationship Diagram	29
3.2	Authentication Flow — JWT + Firebase (Sequence Diagram)	30
3.3	Sprint 1 — RBAC Role Hierarchy	31
3.4	Login Page — Email/Password and Google OAuth2	34
3.5	User Profile Page	34
4.1	Sprint 2 — Asset and Order Entity-Relationship Diagram	37
4.2	Asset Upload Workflow (Sequence Diagram)	38
4.3	Stripe Payment Flow (Sequence Diagram)	39
4.4	Asset Catalog with Category and Price Filters	42
4.5	Interactive 3D Model Viewer	43
4.6	Stripe Payment Checkout Form	43
5.1	Sprint 3 — Reviews, Notifications and Subscriptions ERD	46
5.2	Admin Moderation Use Case Diagram	47
5.3	Admin Moderation Panel — Pending Assets Queue	50
5.4	Notification Center with Unread Badge	50
5.5	Subscription Plans (Pricing Page)	51
6.1	Sprint 4 — Entity-Relationship Diagram	54
6.2	Intern Asset Request Workflow (Sequence Diagram)	55
6.3	Intern Asset Request and Voucher Redemption Page	59
6.4	Peer-to-Peer Asset Trade Interface	59
7.1	IGS Workspace Architecture and Integration with Marketplace	62
7.2	Sprint 5 — Workspace and Chat Entity-Relationship Diagram	63
7.3	Real-Time Chat Architecture with Socket.io	64
7.4	IGS Workspace — Main Dashboard	67

7.5	IGS Workspace — Real-Time Team Chat with Emoji Reactions	67
7.6	IGS Workspace — Task Management	68
8.1	AI Integration Architecture — Server-side LLM API calls	71
8.2	AI-Powered Review Analysis — Seller Dashboard	73
9.1	Two-Stage AI Recommendation Pipeline	77

List of Tables

1.1	Inherited Games Studio — Company Profile	5
1.2	Comparative Analysis of Existing 3D Asset Marketplace Platforms . .	8
1.3	Comparative Analysis of Software Development Methodologies	13
1.4	Sprint Planning Overview	15
1.5	Technology Stack Summary	16
2.1	System Actors and Permission Levels	19
2.2	Non-Functional Requirements	23
2.3	Technology Stack Summary	25
3.1	Sprint 1 — User Stories	28
3.2	Sprint 1 — Test Cases	33
4.1	Sprint 2 — User Stories	36
4.2	Asset Categories and Corresponding Viewer Components	40
4.3	Sprint 2 — Test Cases	41
5.1	Sprint 3 — User Stories	45
5.2	Subscription Plan Tiers	47
5.3	Sprint 3 — Test Cases	49
6.1	Sprint 4 — User Stories	53
6.2	Sprint 4 — Test Cases	58
7.1	Sprint 5 — User Stories	61
7.2	Workspace Sidebar Navigation by Role	63
7.3	Sprint 5 — Test Cases	66
8.1	Sprint 6 — User Stories and Status	70
8.2	AI Model and API Specification	71
9.1	Sprint 7 — User Stories	77
9.2	User Signal Weights for Profile Building	78
9.3	IGS Bot — AI Model Configuration	80
9.4	Upload Form — Key Improvements	81
9.5	New Product Categories Added in Sprint 7	82
9.6	Sprint 7 Seed Assets	83

9.7	Sprint 7 — Database Migrations	84
-----	--	----

List of Acronyms

Acronym	Definition
API	Application Programming Interface
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DB	Database
DM	Direct Message
ERP	Enterprise Resource Planning
GLB	GL Binary (3D format)
GLTF	GL Transmission Format
HDRI	High Dynamic Range Imaging
HTTP	HyperText Transfer Protocol
IGS	Inherited Games Studio
ISITCOM	Institut Supérieur de l'Informatique et des Technologies de Communication
JWT	JSON Web Token
MVC	Model-View-Controller
OAuth	Open Authorization
ORM	Object-Relational Mapping
PBR	Physically Based Rendering
PFE	Projet de Fin d'Études (End of Studies Project)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
SSO	Single Sign-On
UML	Unified Modeling Language
UUID	Universally Unique Identifier

General Introduction

Global Market Context

The global digital content economy is experiencing remarkable expansion. The worldwide 3D model marketplace market was valued at **USD 1.92 billion** in 2023 and is projected to reach **USD 4.86 billion by 2031**, growing at a compound annual growth rate (CAGR) of **11.2%** [15]. This growth is driven primarily by the booming video game industry (accounting for 38.2% of demand), architectural visualization (22.1%), and film & television production (17.8%).

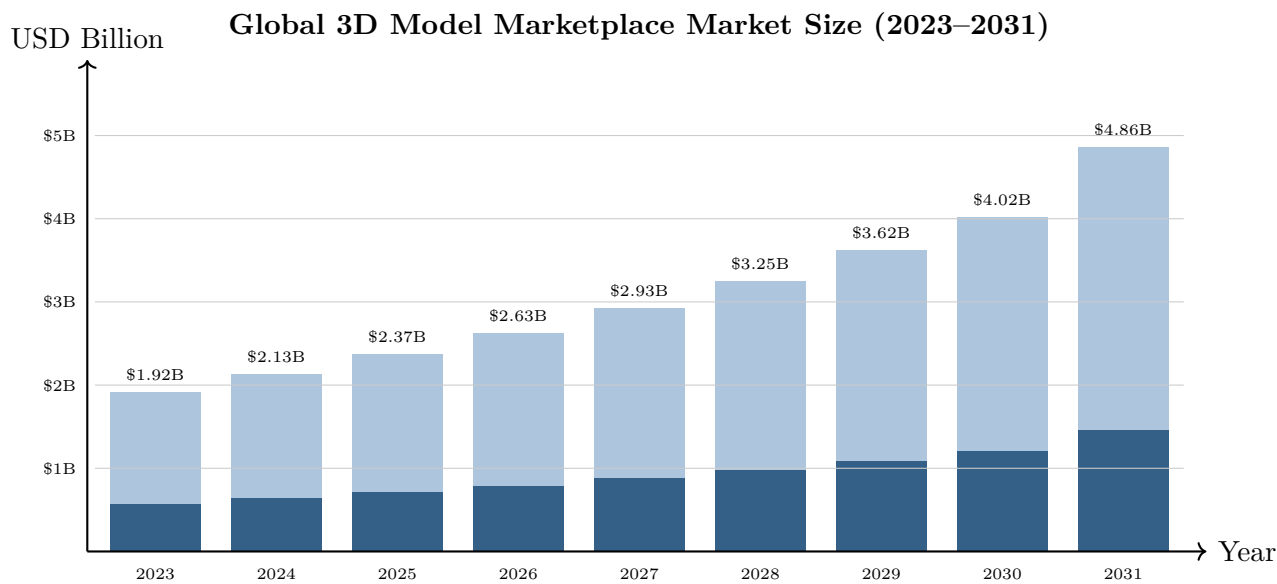


Figure 1: Global 3D model marketplace market growth forecast — CAGR 11.2% (Source: P Market Research, 2024)

The surge in consumer demand for immersive digital content — accelerated by the proliferation of game engines such as Unity and Unreal Engine, the rise of the metaverse, and the democratization of 3D creation tools — has created a vibrant ecosystem of creators and buyers trading digital assets online.

The Tunisian Gap

Despite this global growth, **Tunisia had no dedicated 3D digital asset marketplace** before this project. Tunisian game developers, 3D artists, and interactive media studios were forced to rely entirely on foreign platforms (Unity Asset Store, Sketchfab,

Fab, CGTrader) — platforms that impose dollar-denominated pricing, foreign payment gateways, and policies misaligned with the local creative economy.

Inherited Games Studio (IGS), a Tunisian video game development company, identified this gap and commissioned the development of **IGS Marketplace** — the **first dedicated 3D digital asset marketplace designed and built in Tunisia**. Rather than depending on third-party platforms with limited control over moderation, pricing, and user experience, IGS sought a fully bespoke solution that serves both an internal studio workflow and an external community of creators and buyers.

Project Scope and Mission

This end-of-studies project was carried out within IGS as a six-month internship. The mission was to design and develop, from the ground up, a complete full-stack marketplace platform covering the entire lifecycle of a digital asset: from seller upload and administrative moderation, to buyer discovery, purchase, and download. The project also encompassed a companion internal management application — **IGS Workspace** — designed to support studio teams with task management, project tracking, ERP-style modules, and real-time communication.

The core challenge was to satisfy three distinct user communities simultaneously:

- **External creators** (sellers and buyers) needing a smooth, trustworthy marketplace experience with 3D previews and secure payment;
- **Internal employees and interns** requiring controlled, role-based access to assets and integrated tools;
- **Administrators** demanding full control over content moderation, user management, and platform analytics.

Solution Summary

The developed platform — **IGS Marketplace** — is a modern, full-stack web application built on React, Node.js/Express, and PostgreSQL. It features:

- A **six-level Role-Based Access Control (RBAC)** system covering super-admins, admins, employees, interns, sellers, and members;
- An **interactive 3D asset viewer** with real-time rendering (Three.js + React Three Fiber);
- **Stripe payment integration** with atomic transaction recording;

- A **real-time notification and direct messaging system** (Socket.io);
- An **AI integration layer** using the Anthropic Claude API for intelligent review analysis, content moderation assistance, and an in-app chatbot;
- **Supabase Storage** for scalable asset file hosting;
- A companion **IGS Workspace** dashboard for internal team collaboration.

Report Structure

This report is organized following the **Scrum** agile methodology. Each sprint constitutes a dedicated chapter, documenting the analysis, design, and implementation of its corresponding features:

- **Chapter 1 — General Context:** Host organization, problem statement, comparative study of existing solutions, development methodology, and technology stack.
- **Chapter 2 — Requirements Analysis:** Actors, functional and non-functional requirements, global architecture.
- **Chapter 3 — Sprint 1:** Core infrastructure, database schema design, JWT authentication, OAuth2, and RBAC system.
- **Chapter 4 — Sprint 2:** Asset catalog, interactive 3D viewer, seller upload pipeline, shopping cart, and Stripe payment flow.
- **Chapter 5 — Sprint 3:** Review & rating system, notification center, subscription plans, and admin moderation panel.
- **Chapter 6 — Sprint 4:** Intern role workflows, seller analytics dashboard, asset trading, and community groups.
- **Chapter 7 — Sprint 5:** IGS Workspace companion app — team management, task tracking, project ERP, and real-time chat.
- **Chapter 8 — Sprint 6:** AI integration — review sentiment analysis, content moderation AI, and chatbot assistant.
- **Chapter 9 — Sprint 7:** Advanced AI features — personalized recommendations, Fab-style categorization, and expanded asset taxonomy.
- **General Conclusion:** Summary of achievements, lessons learned, and future perspectives.

Chapter 1

General Context

Introduction

This chapter presents the full context of the project: the host organization, the problem it sought to solve, an in-depth comparative study of existing marketplace solutions worldwide, the proposed solution overview, the development methodology (Scrum/Agile), and the complete technology stack used for development.

1.1 Host Organization: Inherited Games Studio

1.1.1 Company Profile

Inherited Games Studio (IGS) is a Tunisian independent video game development company specializing in the creation of narrative-driven and multiplayer games. The studio brings together artists, developers, designers, and sound engineers passionate about building immersive interactive experiences. IGS also provides creative services and digital content to external clients, positioning itself at the intersection of game development and digital media production.

Table 1.1: Inherited Games Studio — Company Profile

Field	Information
Company Name	Inherited Games Studio (IGS)
Industry	Video Game Development / Digital Content
Location	Tunisia
Team Size	Small studio (< 50 employees)
Specialization	3D game development, digital assets, interactive media
Company Supervisor	Mme. Nour Ltaief

1.1.2 Organizational Structure

IGS is organized around four functional teams:

- **Development Team:** Backend and frontend engineers responsible for engine integration and web services.
- **Art & Design Team:** 3D modelers, texture artists, riggers, and UI/UX designers.
- **Management & Administration:** Project managers, HR, and administrative staff who oversee studio operations.
- **Interns:** Students integrated into active projects under supervised access, contributing to both creative and technical tasks.

1.2 Project Context and Problem Statement

1.2.1 Context

The digital asset industry — encompassing 3D models, PBR textures, rigged characters, game-ready scripts, and environment packs — has undergone massive growth with the democratization of real-time rendering engines such as Unity and Unreal Engine. Studios increasingly outsource or trade asset creation to accelerate production timelines, creating a thriving ecosystem of digital marketplaces.

According to P Market Research (2024), the global **3D model marketplace market** was valued at **USD 1.92 billion** in 2023 and is projected to reach **USD 4.86 billion by 2031**, at a CAGR of **11.2%**. The gaming sector is the primary demand driver, representing **38.2%** of market consumption, followed by architecture (22.1%) and film & television (17.8%).

1.2.2 Problem Statement

Despite operating in this growing market, IGS faced several critical operational pain points before this project:

- **No centralized asset repository:** Assets were scattered across local drives and cloud folders with no unified catalog or search capability.
- **No monetization platform:** IGS had no mechanism to sell or distribute its surplus assets externally, missing a significant revenue stream.
- **No role-based access control:** All internal users shared similar access levels, creating confidentiality and security concerns.
- **Fragmented internal workflows:** Collaboration relied on disconnected tools — chat apps, shared drives, spreadsheets — with no unified project or task management.
- **No structured intern management:** Interns required supervised, limited access to assets and tools, which was not systematically enforced.
- **Dependency on foreign platforms:** Using international platforms (Unity Asset Store, Sketchfab, Fab) meant accepting dollar-denominated fees, foreign payment policies, and no customization.

1.3 Comparative Study of Existing Solutions

1.3.1 The Global 3D Marketplace Landscape

The following platforms represent the primary actors in the international 3D digital asset marketplace space. Each has carved a distinct niche but also carries significant limitations when evaluated against IGS’s specific requirements.

Unity Asset Store

Launched in 2010, the **Unity Asset Store** is the most widely used marketplace for game-ready assets, with over 11,000 publishers and millions of registered users. It offers tight integration with the Unity Editor, a certification program, and a well-established revenue-sharing model (70/30 split). However, it is strictly **Unity-engine specific** — assets must be packaged in `.unitypackage` format, making it inaccessible to developers using Unreal Engine, Godot, or custom pipelines. There is no RBAC, no internal team tooling, and no customization for studio operators.

Fab (Epic Games / Unreal Engine)

Fab (formerly Unreal Marketplace and Quixel Bridge, merged in 2024) is Epic Games' unified content marketplace. It supports multiple engine formats and benefits from deep Unreal Engine integration, MetaHuman compatibility, and the Quixel Megascans library. Despite its impressive scale, Fab is **proprietary and closed** — operators cannot deploy their own instance, cannot customize moderation policies, and the platform collects a 12–30% commission. For a Tunisian studio, payment infrastructure and currency support present additional friction.

Sketchfab (Acquired by Epic, 2021)

Sketchfab pioneered the in-browser 3D viewer and built a community of over 10 million 3D models. It supports GLTF, GLB, FBX, OBJ, and dozens of other formats, and provides an embeddable viewer. However, Sketchfab is primarily a **3D viewing and community platform**, not a specialized marketplace for game assets. There is no internal team management, no content pipeline for studios, no RBAC, and the acquisition by Epic has shifted its roadmap toward Fab integration.

CGTrader

CGTrader is a professional marketplace founded in 2011, focusing on high-quality 3D models for enterprise use including AR/VR e-commerce, industrial design, and architectural visualization. It offers API access for enterprise clients and a Membership program for sellers. CGTrader does not provide a 3D game asset pipeline, lacks internal team tooling, and has no open/self-hosted version. Its primary audience is visualization professionals rather than game developers.

TurboSquid (Shutterstock)

TurboSquid, acquired by Shutterstock in 2021, is one of the oldest 3D marketplaces (founded 2000). It introduced the **CheckMate** quality certification program and serves Hollywood studios, advertising agencies, and enterprise clients. TurboSquid targets premium professional assets at premium prices. It has no gaming-specific pipeline, no internal team features, and its pricing model (sellers earn 40–80% depending on exclusivity) is geared toward professional agencies rather than independent studios or community creators.

Gumroad

Gumroad is a general-purpose digital product marketplace supporting any downloadable file type. Its simplicity and low barrier to entry make it popular among indepen-

dent creators for selling brushes, fonts, ebooks, and occasional 3D assets. However, Gumroad offers **no 3D preview capability**, no content moderation infrastructure, no RBAC, and is not tailored for structured studio operations.

Itch.io

Itch.io is an open indie game and asset marketplace strongly focused on creative freedom and low commissions (developers choose their own cut). It supports asset packs and game jam bundles. Like Gumroad, it lacks a 3D viewer, has no studio-grade moderation, no role hierarchies, and no internal management tools.

1.3.2 Comparative Summary Table

Table 1.2: Comparative Analysis of Existing 3D Asset Marketplace Platforms

Feature	Unity AS	Fab	Sketchfab	CGTrader	Gumroad	IGS MP
3D In-Browser Viewer	Partial	Yes	Yes	No	No	Yes
Multi-Engine Support	No	Yes	Yes	Yes	Yes	Yes
Custom RBAC	No	No	No	No	No	Yes
Internal Team Tools	No	No	No	No	No	Yes
AI Features	No	No	No	No	No	Yes
Self-Hosted / Custom	No	No	No	No	No	Yes
Intern Workflow	No	No	No	No	No	Yes
Real-Time Chat	No	No	No	No	No	Yes
Stripe Payment	No	No	No	No	Yes	Yes
Subscription Plans	No	No	No	Yes	No	Yes
Tunisia- Accessible	Partial	Partial	Partial	Partial	Yes	Yes

1.3.3 Conclusion of the Comparative Study

The analysis confirms a clear gap in the market: **no existing platform combines a 3D digital asset marketplace with internal studio management tools, customizable role hierarchies, and AI-powered features in a single self-hosted solution.** Furthermore, **Tunisia had no dedicated 3D digital asset marketplace before this project** — all platforms analyzed are foreign-owned, impose foreign payment infrastructures, and offer no localization. IGS Marketplace is therefore not merely a custom tool but also **the first 3D digital asset marketplace platform designed and developed in Tunisia**, directly addressing both the studio's internal needs and a broader regional market gap.

1.4 Proposed Solution Overview

The proposed solution — **IGS Marketplace** — is a full-stack web platform composed of two interconnected applications:

1. **IGS Marketplace (port 5173):** The public-facing marketplace where sellers list assets, buyers purchase and download content, and administrators moderate the catalog. It includes a 3D interactive viewer, a Stripe-powered checkout, a subscription system, and an AI chatbot assistant.
2. **IGS Workspace (port 5174):** A private internal dashboard exclusively for IGS employees and interns, featuring team management, task tracking, project ERP modules, a real-time direct messaging system, and role-based navigation.

Both applications share the same backend API (Express.js on port 3001), the same PostgreSQL database, and the same authentication system (Firebase Auth + JWT), enabling seamless Single Sign-On between the two apps.

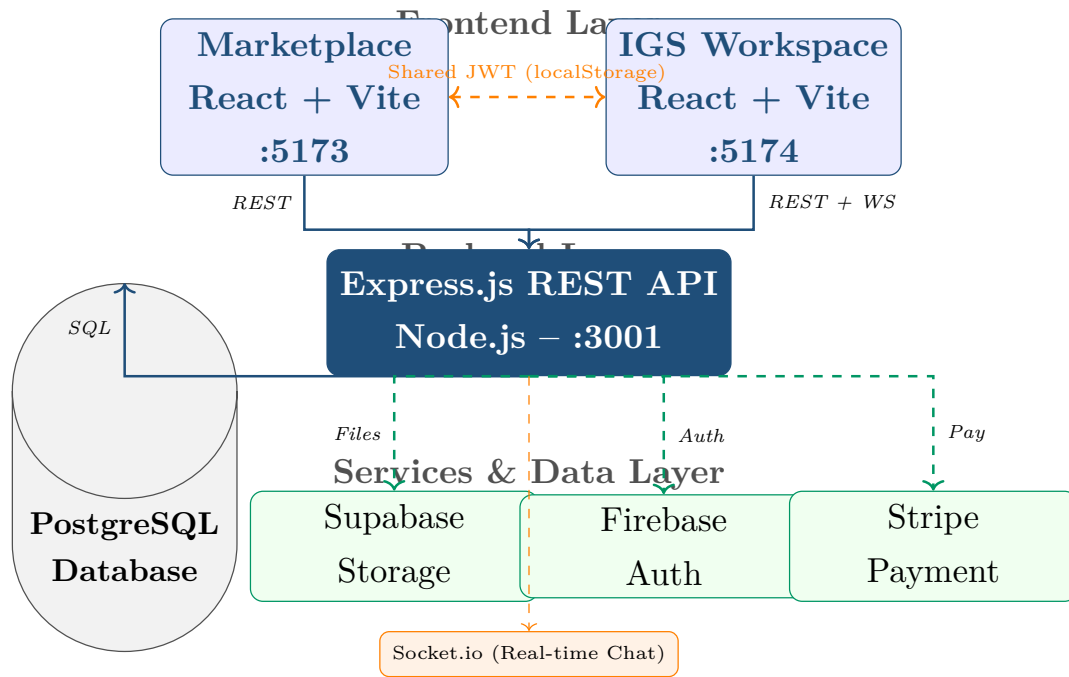


Figure 1.1: Global Architecture of the IGS Platform

1.5 Development Methodology: Scrum / Agile

1.5.1 Comparative Study of Software Development Methodologies

Before adopting a specific methodology, we evaluated the most commonly used approaches in software engineering projects to identify the one best suited to the IGS Marketplace context.

Waterfall Model

The Waterfall model organizes development into strict sequential phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance. Each phase must be fully completed and signed off before the next begins.

Strengths:

- Simple to understand and manage.
- Works well when requirements are fixed and fully known upfront.
- Produces extensive documentation at each phase.

Weaknesses:

- Completely inflexible — a change in requirements after the design phase is extremely costly.

- The working product is only visible at the end, making it impossible to gather feedback during development.
- High risk: if requirements are misunderstood, the error is only discovered at delivery.

Verdict for this project: Incompatible. IGS's requirements emerged progressively throughout the internship. A Waterfall approach would have forced committing to a fixed feature list before any prototyping was done.

Extreme Programming (XP)

Extreme Programming (XP) is an agile methodology focused on technical engineering practices: Test-Driven Development (TDD), pair programming, continuous integration, small frequent releases, and close collaboration with a customer representative.

Strengths:

- High code quality enforced through TDD and continuous integration.
- Very responsive to changing customer requirements.
- Suitable for high-risk, rapidly evolving projects.

Weaknesses:

- Requires pair programming, which assumes a team of at least two developers.
- Heavy reliance on a dedicated, on-site customer representative.
- Can be excessive for projects with a single developer, adding ceremony without benefit.

Verdict for this project: Partially compatible, but the pair programming and on-site customer requirements are not applicable to a one-person internship project.

Kanban

Kanban is a **pull-based** workflow management system derived from Toyota's lean manufacturing practices. Work items are represented as cards on a board and flow through columns (To Do → In Progress → Done). There are no fixed iterations — work is pulled continuously as capacity allows.

Strengths:

- Very lightweight — no ceremonies, no fixed roles.
- Excellent for continuous delivery and maintenance work.

- Work-in-progress limits help identify bottlenecks.

Weaknesses:

- No natural cadence for planning or reviewing — can lead to scope drift.
- Difficult to predict delivery dates for large, complex features.
- Better suited for operational support than for structured feature development.

Verdict for this project: Not ideal for a six-month structured development project with defined deliverables and regular supervisor reviews.

Scrum

Scrum is the most widely adopted agile framework for product development. It organizes work into fixed-length **sprints** (2–4 weeks), each delivering a tested, shippable product increment. Three pillars underpin Scrum: **Transparency**, **Inspection**, and **Adaptation**.

Strengths:

- Iterative delivery: a working increment is produced at the end of every sprint.
- Regular reviews with the Product Owner allow continuous feedback and course correction.
- The Sprint Backlog allows new features to be reprioritized between sprints.
- Defined roles and ceremonies provide structure without excessive overhead.
- Well-adapted to single-developer projects when the Product Owner and Scrum Master roles are played by supervisors.

Weaknesses:

- Sprint planning and retrospectives add meeting overhead.
- Scope can creep if the Product Backlog is not well-maintained.

Verdict for this project: Chosen. Scrum is the best fit because requirements emerged progressively, supervisors (company and academic) act naturally as Product Owner and Scrum Master, and the sprint-based structure aligns with university internship review cycles.

Summary Comparison Table

Table 1.3: Comparative Analysis of Software Development Methodologies

Criterion	Waterfall	XP	Kanban	Scrum
Flexible requirements	No	Yes	Yes	Yes
Structured delivery cadence	Yes	Yes	No	Yes
Solo developer compatible	Yes	No	Yes	Yes
Regular supervisor reviews	No	Partial	No	Yes
Risk management	Low	High	Medium	High
Suitable for internship format	No	Partial	Partial	Yes

1.5.2 Why Scrum?

Based on the comparative analysis above, **Scrum** was selected as the development methodology for the following reasons:

1. **Progressive requirements:** IGS’s feature priorities evolved throughout the internship. Scrum’s backlog-based planning allowed inserting and reprioritizing features between sprints without disrupting ongoing work.
2. **Supervisor alignment:** The company supervisor (Mme. Nour Ltaief) naturally assumes the *Product Owner* role — defining priorities and validating deliverables. The academic supervisor (Mr. Sami Ben Amor) naturally assumes the *Scrum Master* role — ensuring process quality and removing obstacles.
3. **Regular validation points:** Sprint Reviews align with the mandatory internship progress meetings, creating a natural rhythm of demonstration and feedback every 2–3 weeks.
4. **Risk reduction:** Delivering working software at the end of every sprint means that even if the project is cut short, a functional product exists at every stage.

1.5.3 The Scrum Framework

Scrum organizes work into fixed-length iterations called **sprints** (2–4 weeks), each producing a potentially shippable product increment. Three pillars define Scrum:

Transparency (all work is visible), **Inspection** (progress is regularly examined), and **Adaptation** (plans are adjusted based on inspection results).

Scrum Roles

- **Product Owner:** Mme. Nour Ltaief (Company Supervisor) — defines and prioritizes the Product Backlog based on business value and studio needs.
- **Scrum Master:** Mr. Sami Ben Amor (Academic Supervisor) — ensures the Scrum process is correctly followed and helps remove impediments.
- **Development Team:** Asma Daoussi (student developer) — solely responsible for designing, developing, and delivering each sprint increment.

Scrum Ceremonies

- **Sprint Planning:** At the start of each sprint, user stories are selected from the Product Backlog and broken into technical tasks with effort estimates.
- **Daily Scrum:** A brief daily check-in to track progress against the sprint goal and identify blockers.
- **Sprint Review:** At the end of each sprint, the implemented features are demonstrated to the Product Owner and validated against acceptance criteria.
- **Sprint Retrospective:** A reflection session to evaluate what worked, what did not, and what process improvements to apply in the next sprint.

Scrum Artifacts

- **Product Backlog:** The master list of all desired features, user stories, and improvements, ordered by priority.
- **Sprint Backlog:** The subset of the Product Backlog selected for the current sprint, plus the plan for delivering it.
- **Product Increment:** The sum of all completed Product Backlog items, forming a usable version of the product at the end of each sprint.

1.5.4 Sprint Planning Overview

The project was divided into **seven sprints**, each lasting approximately two to three weeks:

Table 1.4: Sprint Planning Overview

Sprint	Theme	Key Deliverables
1	Core Infrastructure & Authentication	Backend setup, PostgreSQL schema, JWT, Google OAuth2, RBAC (6 roles)
2	Asset Marketplace Core	Asset catalog, 3D viewer, seller upload, shopping cart, Stripe payment
3	Content Quality & Administration	Reviews & ratings, AI summaries, notifications, admin panel, subscriptions
4	Advanced Role Features	Intern workflows, seller analytics, asset trading, community groups
5	Internal Workspace (IGS Workspace)	Companion app, task management, project ERP, real-time chat (Socket.io)
6	AI Integration (Phase 1)	Anthropic Claude API, review sentiment, moderation assistance, chatbot
7	AI Features & Expanded Categories	Recommendation engine, advanced chatbot, Fab-style taxonomy (12+ categories)

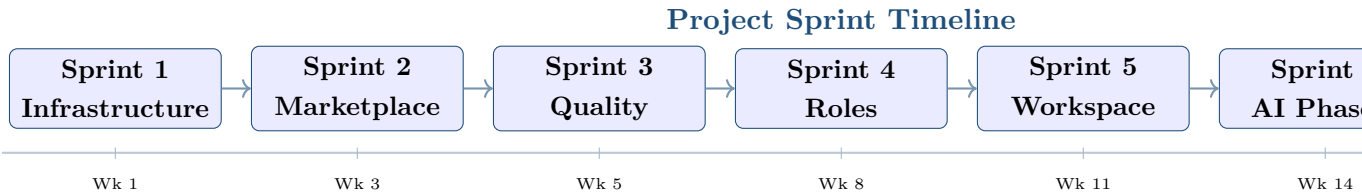


Figure 1.2: Sprint timeline overview — 7 sprints over 20 weeks

1.6 Development Environment and Technology Stack

This section presents the complete set of technologies, frameworks, and tools used throughout the project.

1.6.1 Technology Stack Overview

Table 1.5: Technology Stack Summary

Layer	Technology	Role / Justification
Frontend	React 18 + Vite	SPA framework — fast HMR, component model
	Three.js / R3F	3D asset viewer in the browser
	TailwindCSS	Utility-first responsive styling
Backend	Node.js + Express.js	REST API server — non-blocking, JSON-native
	Socket.io	WebSocket real-time events (chat, notifications)
	Anthropic Claude API	AI review analysis, chatbot, recommendations
Database & Storage	PostgreSQL 16	Primary relational database
	Supabase Storage	Scalable file hosting for 3D assets and thumbnails
	Firebase Auth	Google OAuth2 + email/password authentication
Payments	Stripe Stripe.js (client SDK)	Secure card payment processing Client-side payment intent confirmation
Dev Tools	Git + GitHub	Version control and code repository
	Visual Studio Code	Primary code editor
	Postman	API endpoint testing
	pgAdmin / DBeaver	PostgreSQL administration

1.6.2 Technology Badges

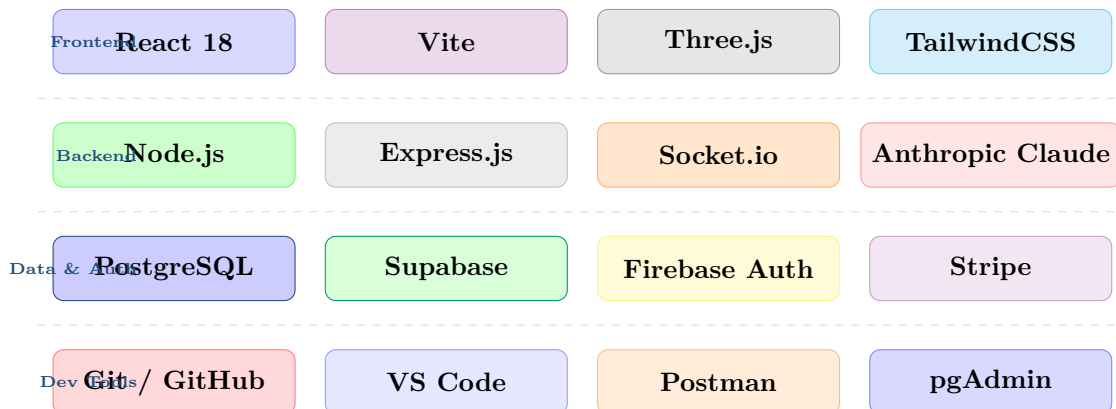


Figure 1.3: Technology stack used across frontend, backend, data layer, and tooling

1.6.3 Key Technology Choices

React + Vite were chosen for the frontend due to their speed (Vite’s ES module-based HMR), React’s mature component ecosystem, and the availability of libraries such as React Three Fiber for 3D rendering and Stripe.js for payment UI.

Node.js + Express.js power the REST API backend due to their non-blocking I/O model — particularly suited for handling concurrent file upload requests and WebSocket events without thread-blocking overhead.

PostgreSQL was selected over NoSQL alternatives because the domain model (users, assets, orders, reviews, notifications, subscriptions) is inherently relational, and PostgreSQL’s ACID guarantees are essential for payment transaction integrity.

Firebase Authentication handles identity (Google OAuth2 and email/password) while delegating authorization entirely to the backend RBAC layer — providing a clean separation between authentication and business-logic-level permissions.

Supabase Storage provides scalable, CDN-backed object storage for 3D asset files (GLB, FBX, OBJ) and thumbnails, with public URL generation — eliminating the complexity of managing a custom file server.

Anthropic Claude API (model: `claude-haiku-4-5-20251001`) powers the AI layer: review sentiment analysis, moderation assistance, and the in-app chatbot. Claude Haiku was chosen for its balance of intelligence and low latency at minimal cost.

Stripe is the industry-standard payment processor, offering PCI-compliant card processing, a client-side Stripe.js SDK for secure card capture, and a robust PaymentIntent API for atomic transaction recording.

Conclusion

This chapter has established the full context of the project: the host organization IGS and its operational challenges, the global market opportunity (a USD 4.86B industry by 2031), a detailed comparative analysis of international platforms revealing the absence of any dedicated 3D marketplace in Tunisia, the proposed two-application solution, the Scrum agile methodology applied across seven sprints, and the technology stack chosen for its scalability and suitability to the domain. The following chapter details the functional and non-functional requirements of the system.

Chapter 2

Requirements Analysis

Introduction

This chapter provides a thorough analysis of the system requirements. We identify the actors involved, enumerate functional and non-functional requirements, present the global use case diagram, and define the technology stack.

2.1 Actors Identification

The IGS Marketplace platform serves six distinct actor types, organized in a hierarchical Role-Based Access Control (RBAC) system:

Table 2.1: System Actors and Permission Levels

Role	Type	Level	Key Permissions
super_admin	Internal	100	Full system access, manage roles, system settings
admin	Internal	80	Manage users, assets, orders, view analytics
employee	Internal	60	Moderate content, view all analytics, manage workspace
intern	Internal	40	Browse catalog, request free assets, view assigned tasks
seller	External	20	Upload assets, manage own listings, view own analytics
member	External	10	Purchase assets, add to cart, download, write reviews

2.1.1 Actor Descriptions

Super Administrator The platform owner with unrestricted access to all features. Can manage roles, system configuration, and has full oversight of every entity in

the system.

Administrator Responsible for day-to-day platform management: approving or rejecting submitted assets, managing user accounts, processing seller requests, and accessing platform-wide analytics.

Employee An IGS staff member who can moderate content and access reporting tools. Has access to the IGS Workspace for internal collaboration.

Intern A student or trainee within IGS. Can browse the asset catalog and request access to specific assets (subject to supervisor approval). Has access to workspace tasks assigned to them.

Seller An external creator who has been granted the seller role (either directly or via a request). Can upload, edit, and manage their own digital assets.

Member A standard registered external user. Can browse the catalog, add items to cart, purchase and download assets, write reviews, and participate in community groups.

2.2 Functional Requirements

2.2.1 Authentication & User Management

- Users can register via email/password or via OAuth2 (Google, GitHub).
- Users can log in and receive a JWT for API authentication.
- Users can view and edit their profile (username, bio, avatar).
- Admins can ban, activate, or change the role of any user.
- External users can request the seller role from the admin panel.

2.2.2 Asset Catalog

- Any visitor can browse the public asset catalog.
- Assets can be filtered by category, price range, rating, and free/paid status.
- Assets can be searched by title or keyword (tag-based search).
- Each asset has a detail page with description, metadata, previews, and a 3D/HDRI/texture viewer.
- The system supports 7 asset categories: 3D Models, Textures, Scripts, Shaders, HDRIs, Plugins, Printables.

2.2.3 Asset Upload & Management (Seller)

- Sellers can upload assets with title, description, category, price, tags, thumbnail, and files.
- Sellers can upload multiple format variants per asset (e.g., GLB, FBX, OBJ for the same 3D model).
- Submitted assets enter a *pending* state awaiting admin review.
- Sellers can submit edit requests for approved assets.
- Sellers have access to a personal analytics dashboard (views, downloads, revenue).

2.2.4 Shopping Cart & Orders

- Authenticated users can add assets to a persistent shopping cart.
- Users can proceed to checkout and pay via Stripe.
- Orders are recorded with status tracking (pending, paid, refunded, cancelled).
- After a successful payment, the asset is added to the user's library.
- Free assets can be claimed without payment.

2.2.5 Reviews & Ratings

- Users who own an asset (via purchase or free download) can submit a review with a rating (1–5 stars).
- Reviews display the author, rating, comment, and date.
- Any user can like/heart an asset.
- The asset's average rating and review count are automatically updated.

2.2.6 Admin Panel

- Admins can view all pending assets and approve or reject them with a reason.
- Admins can manage all user accounts (view, ban, change role).
- Admins can view platform-wide analytics (total sales, downloads, top assets).
- Admins can manage seller requests from external users.
- All admin actions are logged in an audit trail.

2.2.7 Notifications

- Users receive real-time and persistent notifications for key events (asset approved/rejected, order confirmed, new review, etc.).
- Notifications are marked as read individually or all at once.
- A notification bell icon in the navbar shows the unread count.

2.2.8 Subscriptions

- The platform offers three subscription tiers: Free, Subscriber, and Seller.
- Each tier grants different download limits and access levels.
- Users can upgrade their subscription plan.

2.2.9 Intern Role Features

- Interns can browse the catalog and request access to specific paid assets.
- Admins can approve or reject intern asset requests and optionally generate access vouchers.
- Interns can redeem vouchers to access assets without payment.
- Interns can submit assets under supervision (supervised upload workflow).

2.2.10 Community Features

- Users can create and join community groups by category.
- Group members can discuss assets and share resources.
- The platform supports P2P asset trading between users.

2.2.11 IGS Workspace

- Internal users access a separate dashboard app (IGS Workspace).
- The workspace features: team management, task tracking, project management (ERP), and real-time chat.
- Chat supports channels, direct messages, emoji reactions, and read receipts (powered by Socket.io).
- Navigation is role-based: interns see fewer sections than admins.

2.3 Non-Functional Requirements

Table 2.2: Non-Functional Requirements

Category	Requirements
Performance	API response time under 300ms for standard queries; asset file uploads streamed directly to cloud storage; database queries optimized with indexes.
Security	JWT tokens with expiration; Firebase ID token verification on all protected routes; RBAC middleware on every sensitive endpoint; SQL injection prevention via parameterized queries; CORS policy; HTTPS in production.
Scalability	Stateless REST API architecture; cloud file storage (Supabase); database connection pooling; horizontal scaling ready.
Usability	Responsive UI for desktop and tablet; intuitive navigation; loading states and error messages on all user-facing operations.
Reliability	Centralized error handling middleware; database transactions for critical operations (orders); audit logging for admin actions.
Maintainability	Clean separation: routes → controllers → repositories; modular React components; consistent naming conventions throughout.

2.4 Global Use Case Diagram

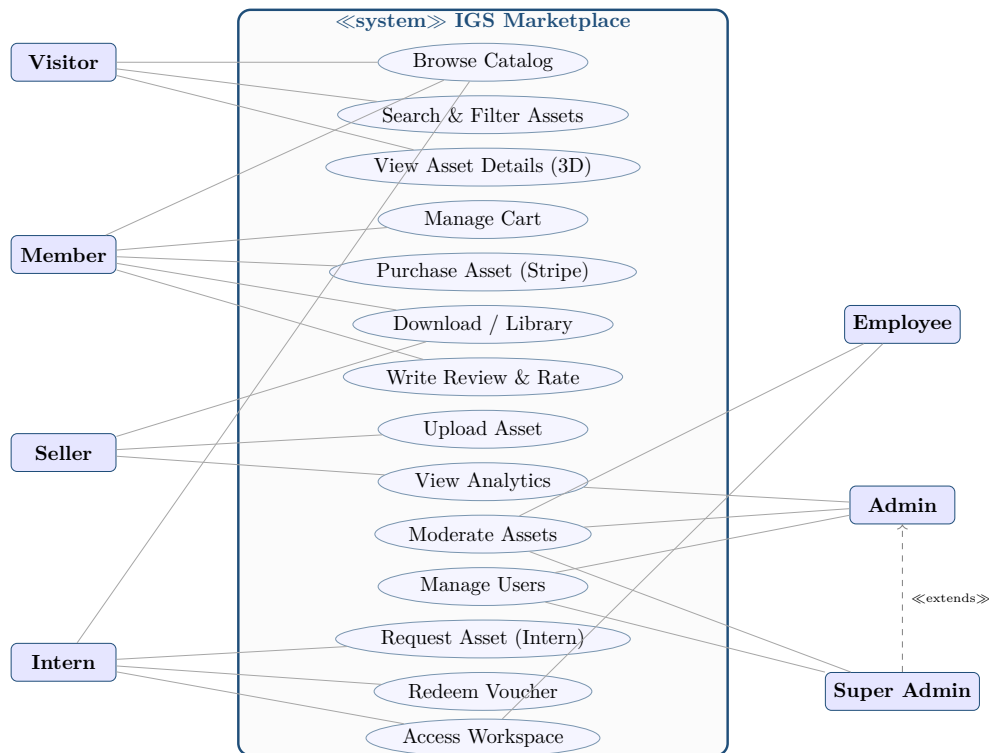


Figure 2.1: Global Use Case Diagram

2.5 System Architecture

2.5.1 Overall Architecture

The IGS platform follows a **client-server architecture** with clear separation of concerns:

- **Frontend (React SPA):** Two separate React applications (Marketplace & Workspace) communicate with the backend exclusively via RESTful API calls. Both share the same JWT stored in `localStorage`.
- **Backend (Express.js REST API):** A single Express server handles all business logic through a layered architecture: Routes → Controllers → Repositories → PostgreSQL.
- **Database (PostgreSQL):** A relational PostgreSQL database stores all platform data, organized across 20+ tables with UUID primary keys and proper foreign key constraints.

- **Cloud Storage (Supabase):** Asset files (GLB, textures, scripts) and thumbnails are stored on Supabase Storage, with signed URLs for secure access.
- **Authentication (Firebase):** Firebase handles identity management (email/password, Google, GitHub OAuth). The backend verifies Firebase ID tokens on every protected request.
- **Real-time (Socket.io):** The chat system in IGS Workspace uses Socket.io for bi-directional real-time messaging.
- **Payment (Stripe):** Stripe PaymentIntents handle card payments securely without storing card data on the server.

2.5.2 Technology Stack

Table 2.3: Technology Stack Summary

Layer	Technology	Purpose
Frontend Framework	React 18 + Vite	SPA with fast HMR development server
UI Components	Tailwind CSS + lucide-react	Utility-first styling, icons
3D Viewer	Three.js / @google/model-viewer	Interactive GLB/GLTF/HDRI preview
State Management	React Context API	Auth, permissions, cart state
Backend Framework	Express.js (Node.js)	REST API server
Database	PostgreSQL 16	Relational data storage
ORM / Query	pg (node-postgres)	Raw SQL with parameterized queries
Authentication	Firebase Auth + JWT	Identity management + API auth
Cloud Storage	Supabase Storage	Asset files and thumbnails
Payment	Stripe	Secure card payments
Real-time	Socket.io	Chat and live notifications
AI (Sprint 6)	Anthropic Claude API	Review analysis, chatbot
Version Control	Git + GitHub	Source code management

2.5.3 Backend Layered Architecture

The backend follows a strict three-layer pattern:

1. **Routes layer** (`src/routes/`): Defines API endpoints, applies authentication and permission middlewares, and delegates to controllers.
2. **Controller layer** (`src/controllers/`): Validates request data, orchestrates business logic, calls repository methods, and formats HTTP responses.
3. **Repository layer** (`src/repositories/`): Contains all SQL queries, encapsulating database access and returning plain objects.

This pattern ensures testability, separation of concerns, and easy maintenance.

Conclusion

This chapter has identified all actors, detailed the functional and non-functional requirements, and established the architectural foundation of the IGS Marketplace. The following chapters each cover one sprint of the development cycle, presenting the backlog, design, implementation, and testing for each set of features.

Chapter 3

Sprint 1: Platform Foundation

Introduction

Sprint 1 establishes the technical foundation upon which the entire platform is built. By the end of this sprint, a fully operational Express.js server with PostgreSQL database, a complete authentication system (JWT + OAuth2), and a Role-Based Access Control layer were deployed. The React frontend skeleton was also initialized with authentication and permissions contexts.

3.1 Sprint Backlog

Sprint 1 — Duration: 3 weeks

Goal: Deliver a fully functional authentication system with RBAC on a clean backend and database foundation.

Table 3.1: Sprint 1 — User Stories

ID	User Story	Priority	Points
US-01	As a user, I want to register with email and password so that I can create an account.	High	3
US-02	As a user, I want to log in with Google so that I can use Single Sign-On.	High	5
US-03	As a user, I want to receive a JWT so that I can authenticate my API requests.	High	3
US-04	As an admin, I want every API endpoint protected by role-based permissions so that unauthorized access is prevented.	High	8
US-05	As a developer, I need a PostgreSQL database schema so that all platform data is structured and persistent.	High	5
US-06	As a user, I want to view and edit my profile (username, bio, avatar).	Medium	3
US-07	As an admin, I want all user actions to be recorded in an audit trail.	Medium	3
Total Story Points:			30

3.2 Analysis and Design

3.2.1 Database Schema — Core Tables

The foundational database schema defines four core entity groups: `categories`, `users`, RBAC permission tables, and `audit_logs`.

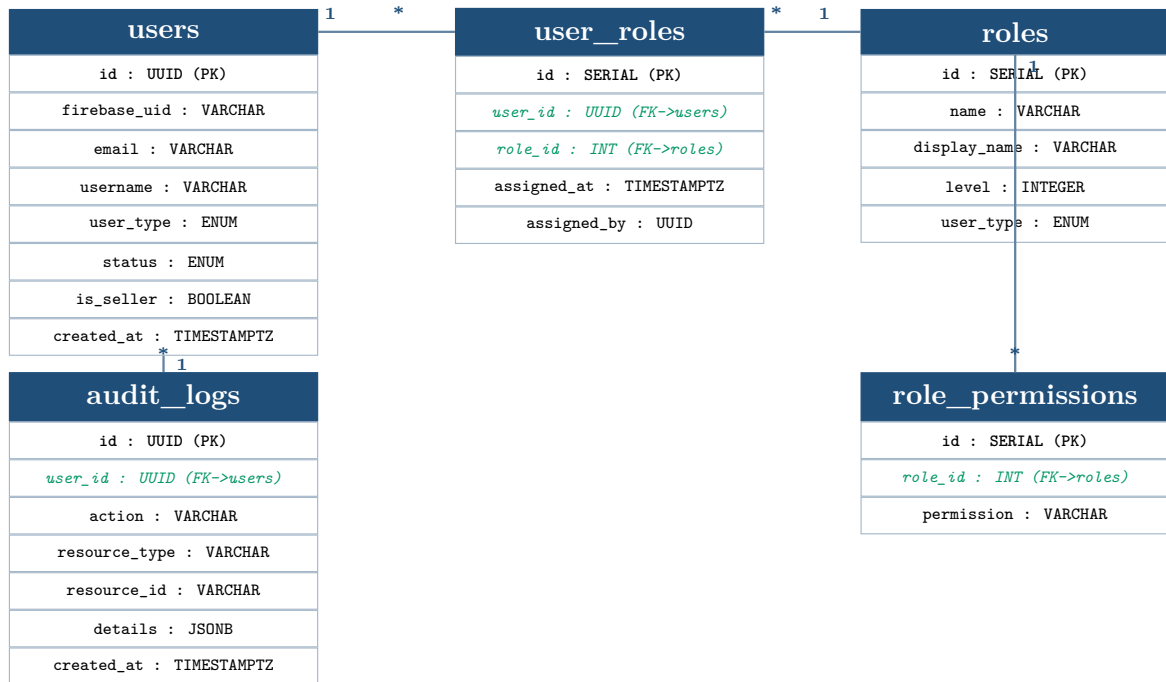


Figure 3.1: Sprint 1 — Core Entity-Relationship Diagram

The **users** table uses a UUID primary key and stores a **firebase_uid** to link each record to the Firebase identity provider. A **user_type** enum distinguishes between **internal** (employees, interns) and **external** (sellers, members) users.

3.2.2 Authentication Flow

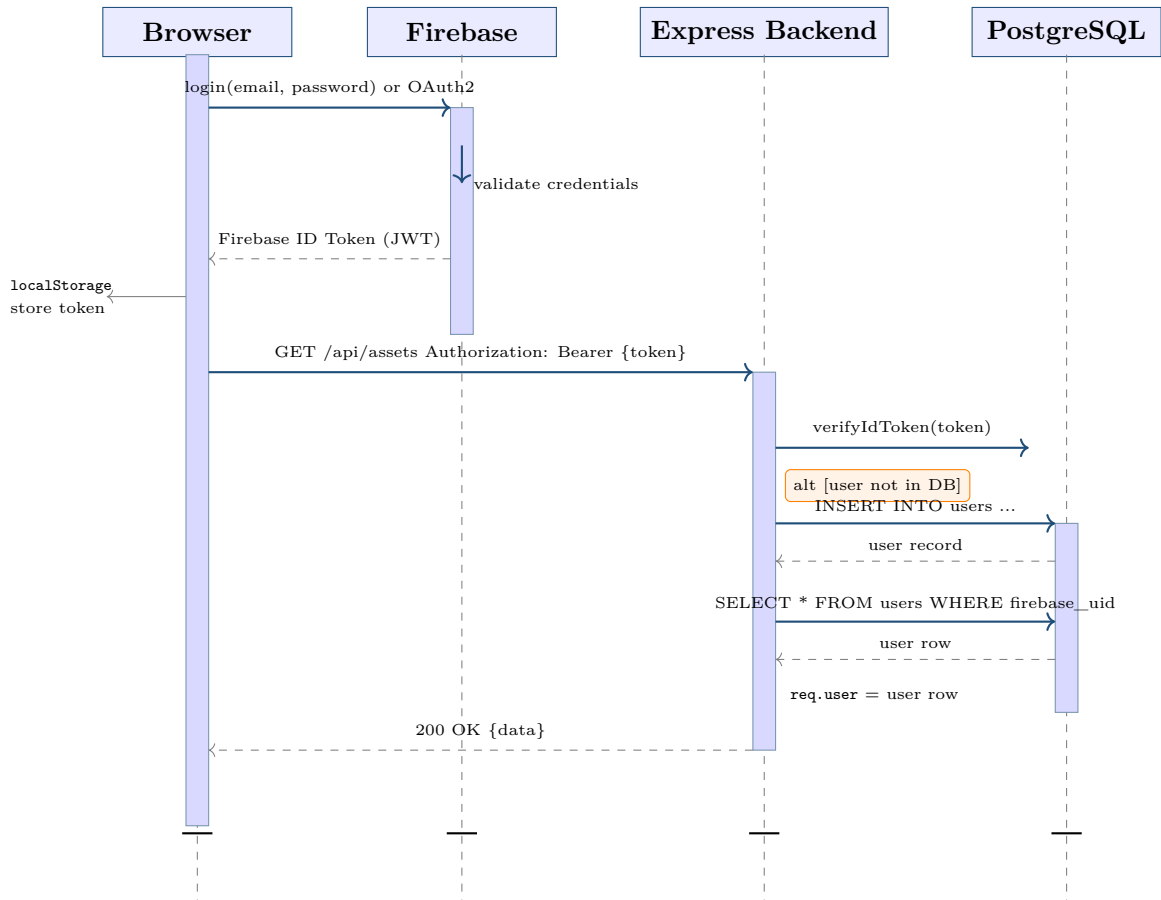


Figure 3.2: Authentication Flow — JWT + Firebase (Sequence Diagram)

The authentication flow operates as follows:

1. The client authenticates with Firebase (email/password or Google OAuth2).
2. Firebase returns a signed ID token (JWT).
3. The client sends the token in the **Authorization: Bearer** header on every request.
4. The backend middleware verifies the token using the Firebase Admin SDK.
5. If valid, the middleware fetches the corresponding user record from PostgreSQL, creating it automatically on first login.
6. The request proceeds with the authenticated user context populated.

3.2.3 RBAC Architecture

The permission system is built on two dedicated database tables: `user_roles` and `role_permissions`. A custom middleware intercepts every protected route and checks whether the authenticated user's role holds the required permission before granting access.

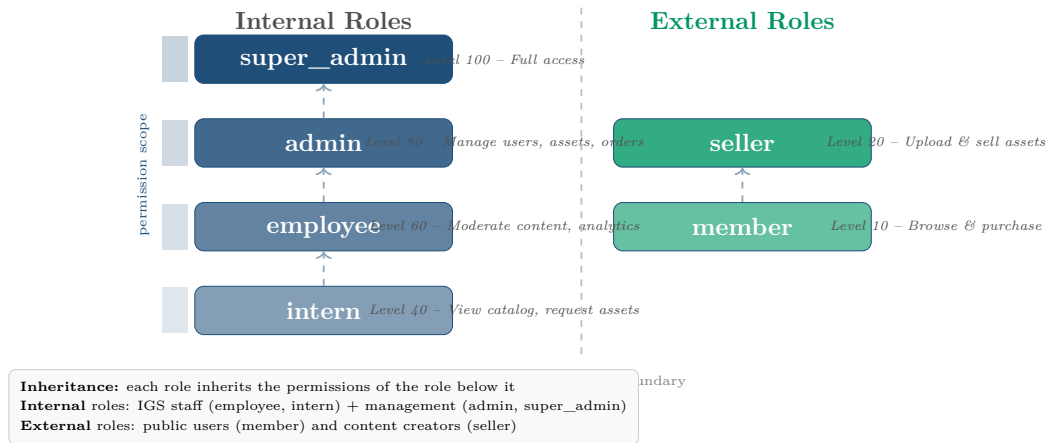


Figure 3.3: Sprint 1 — RBAC Role Hierarchy

The six roles are ordered by privilege level: *super_admin* (100) > *admin* (80) > *employee* (60) > *intern* (40) > *seller* (20) > *member* (10). Higher-privilege roles inherit all permissions of lower ones.

3.3 Implementation

3.3.1 Backend Infrastructure

The Express.js server runs on port 3001 and is configured with:

- **CORS:** Configured to accept requests from both frontend origins (ports 5173 and 5174).
- **Body parsing:** JSON and URL-encoded request bodies, with a 50 MB limit for upload metadata.
- **Centralized error handling:** A global middleware catches all unhandled errors and returns structured JSON responses with appropriate HTTP status codes.
- **Database connection pool:** A PostgreSQL connection pool manages concurrent database connections efficiently.

3.3.2 Authentication Middleware

The `authenticate` middleware is applied to all protected routes. It extracts the Bearer token from the request header, verifies it with Firebase Admin SDK, and retrieves the corresponding user record from PostgreSQL. If the user does not yet exist in the database (first login), a record is automatically provisioned with the default *member* role. This mechanism ensures that every registered Firebase user has a corresponding platform profile.

3.3.3 RBAC Permission Middleware

The `requirePermission` middleware takes a permission identifier as its argument and is stacked on any route that requires authorization. It queries the `role_permissions` table to verify that the authenticated user's role holds the required permission. If not, a 403 Forbidden response is returned. This design allows fine-grained control: for example, `requirePermission('manage_assets')` protects all asset moderation endpoints.

3.3.4 Frontend Authentication Context

On the React side, an `AuthContext` wraps the entire application. It monitors the Firebase authentication state, stores the ID token in `localStorage`, and fetches the user's full profile (including role and permissions) from the backend. A companion `PermissionsContext` exposes a `can(permission)` helper function that React components use to conditionally render UI elements based on the user's role.

3.4 Testing

Table 3.2: Sprint 1 — Test Cases

TC	Test Case	Expected Result	Status
TC-01	Register with valid email/password	User created in Firebase and PostgreSQL	PASS
TC-02	Login with Google OAuth	Token returned, user auto-provisioned in DB	PASS
TC-03	API call without token	401 Unauthorized	PASS
TC-04	API call with expired token	401 Invalid token	PASS
TC-05	Member accesses admin endpoint	403 Forbidden	PASS
TC-06	Admin accesses admin endpoint	200 OK	PASS
TC-07	Update profile (username, bio)	User record updated, changes reflected in UI	PASS
TC-08	Admin action performed	Audit log entry created with user, action, timestamp	PASS

3.5 Sprint Review

3.5.1 Deliverables

By the end of Sprint 1, the following components were delivered and validated:

- Express.js server with full middleware stack (CORS, body parser, error handler, audit logger).
- PostgreSQL database with core schema (`users`, `categories`, `audit_logs`) and RBAC permission tables.
- Firebase Admin SDK integration for token verification.
- Email/password and Google OAuth2 authentication flows.
- RBAC middleware protecting all sensitive API endpoints.

- React frontend with `AuthContext`, `PermissionsContext`, and `ProtectedRoute` component.
- User profile page (view and edit with avatar upload).

3.5.2 Sprint Screenshots

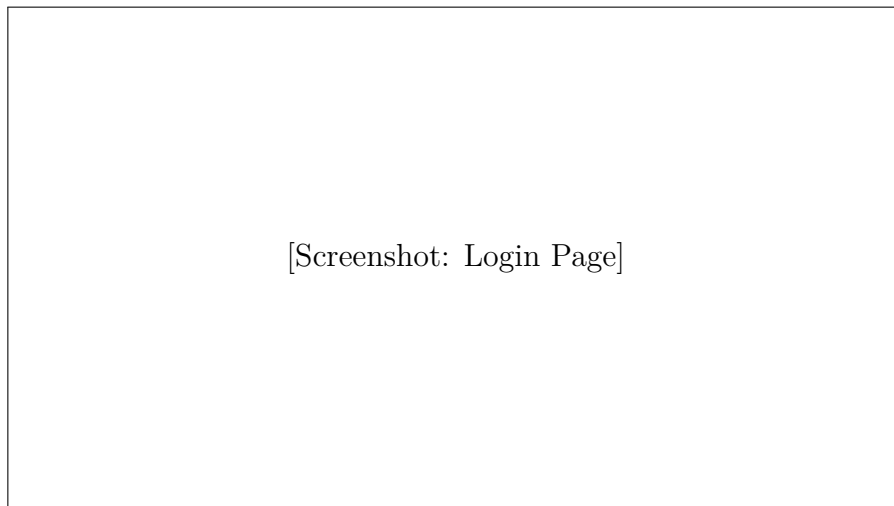


Figure 3.4: Login Page — Email/Password and Google OAuth2

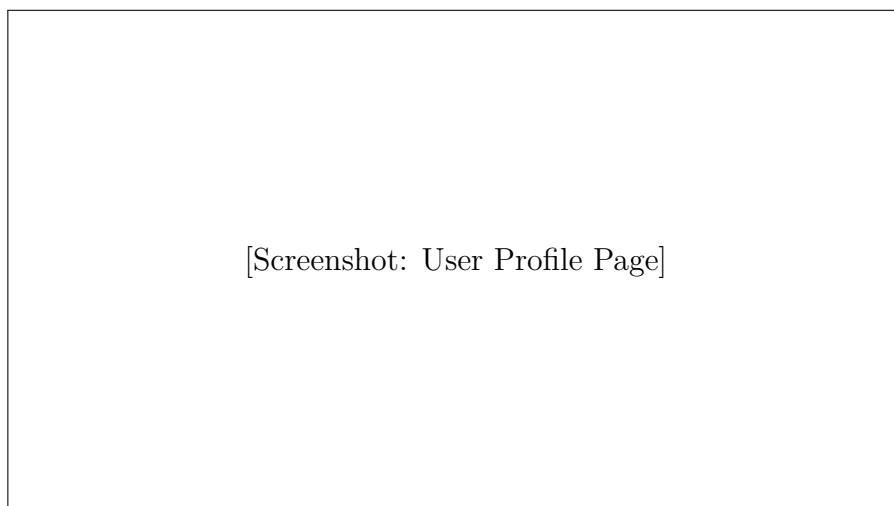


Figure 3.5: User Profile Page

Conclusion

Sprint 1 successfully delivered the foundational infrastructure on which all subsequent features are built. The authentication system, RBAC layer, and database schema are stable and extensible. Sprint 2 will build upon this foundation to implement the core asset marketplace functionality.

Chapter 4

Sprint 2: Marketplace Core Features

Introduction

Sprint 2 delivers the heart of the platform: the asset catalog, the interactive 3D viewer, the seller upload workflow, the shopping cart, and Stripe payment integration. By the end of this sprint, users can browse, purchase, and download digital assets through a complete end-to-end e-commerce flow.

4.1 Sprint Backlog

Sprint 2 — Duration: 3 weeks

Goal: Deliver a functional end-to-end marketplace — from asset listing to purchase and download.

Table 4.1: Sprint 2 — User Stories

ID	User Story	Priority	Points
US-08	As a visitor, I want to browse the asset catalog with filters so that I can find relevant assets.	High	5
US-09	As a visitor, I want to view an asset detail page with a 3D preview so that I can evaluate quality before purchasing.	High	8
US-10	As a seller, I want to upload an asset with files and metadata so that it can be reviewed and listed.	High	8
US-11	As a member, I want to add assets to my cart so that I can manage my purchases.	High	3
US-12	As a member, I want to pay securely with Stripe so that I can complete my purchase.	High	8
US-13	As a member, I want to access my library so that I can re-download assets I own.	High	3
US-14	As a member, I want to claim free assets without payment so that I can build my library at no cost.	Medium	2
US-15	As a seller, I want to upload multiple file formats per asset (GLB, FBX, OBJ).	Medium	5
Total Story Points:			42

4.2 Analysis and Design

4.2.1 Database Schema — Asset and Commerce Tables

Sprint 2 extends the database with the asset-related and commercial transaction tables:

- **assets:** Core asset record with seller reference, category, price, publication status, and a JSONB `metadata` column for category-specific attributes.
- **asset_tags:** Many-to-many tag associations per asset for search and filtering.
- **asset_formats:** Multiple downloadable format entries per asset (from migration 015).
- **orders & order_items:** Order header and line items, with a price-at-purchase snapshot to preserve historical accuracy.
- **cart_items:** Persistent cart stored in the database rather than in session or cookies, ensuring cross-device consistency.

- **user_downloads:** Ownership records that track which user has the right to download which asset.

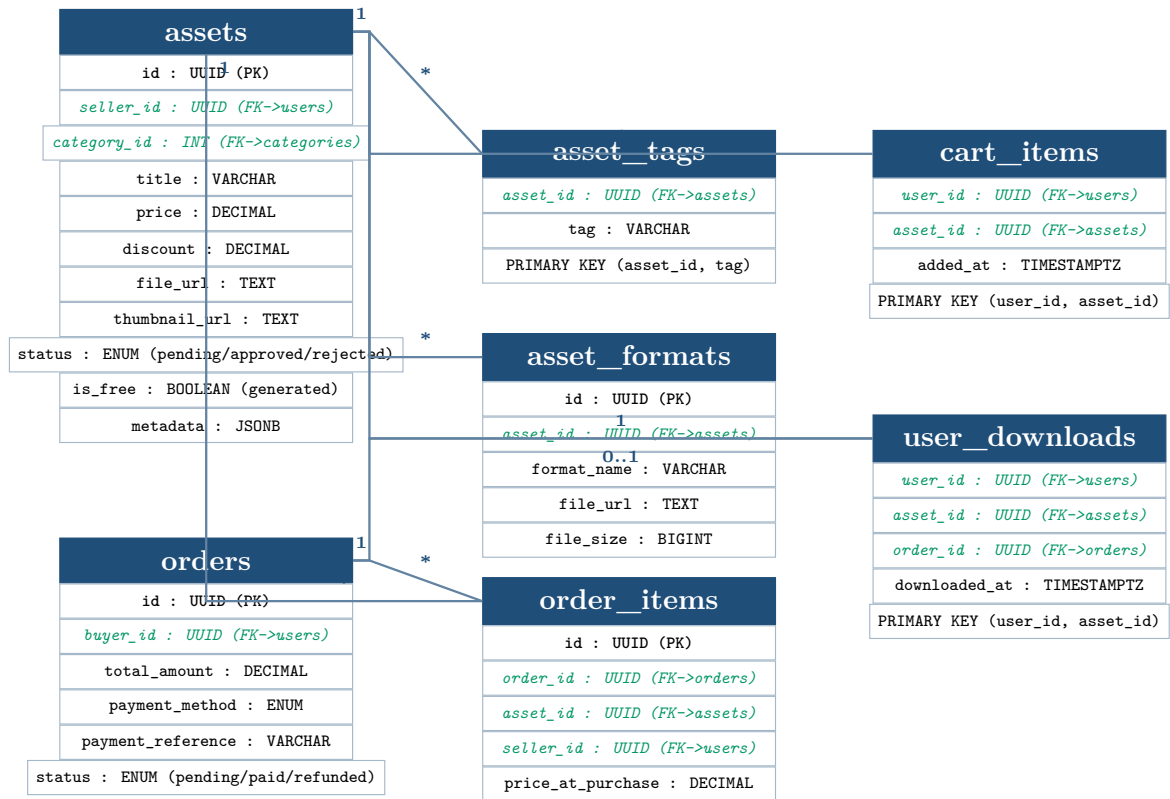


Figure 4.1: Sprint 2 — Asset and Order Entity-Relationship Diagram

The **assets** table includes a computed column **is_free** (derived from **price** = 0) to simplify free-asset queries, and a JSONB **metadata** column for category-specific fields (e.g., polygon count for 3D models, map types for texture packs).

4.2.2 Asset Upload Workflow

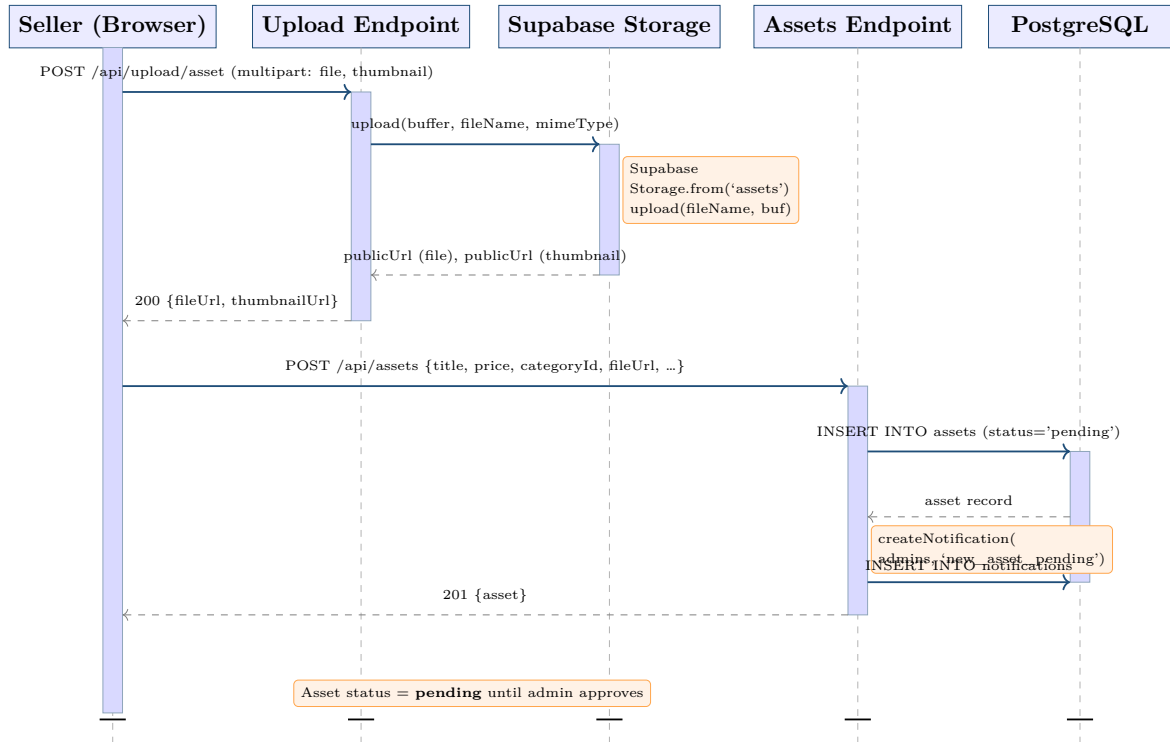


Figure 4.2: Asset Upload Workflow (Sequence Diagram)

The upload flow is divided into two phases:

1. **File upload phase:** The seller selects the asset file and a thumbnail. The frontend posts them to a dedicated upload endpoint, which stores the files in Supabase Storage and returns the public CDN URLs.
2. **Metadata creation phase:** The frontend submits all asset metadata (title, description, price, category, tags) along with the file URLs from the previous step. The backend creates the asset record with status **pending** and notifies all admins of the new submission.

4.2.3 Payment Flow — Stripe

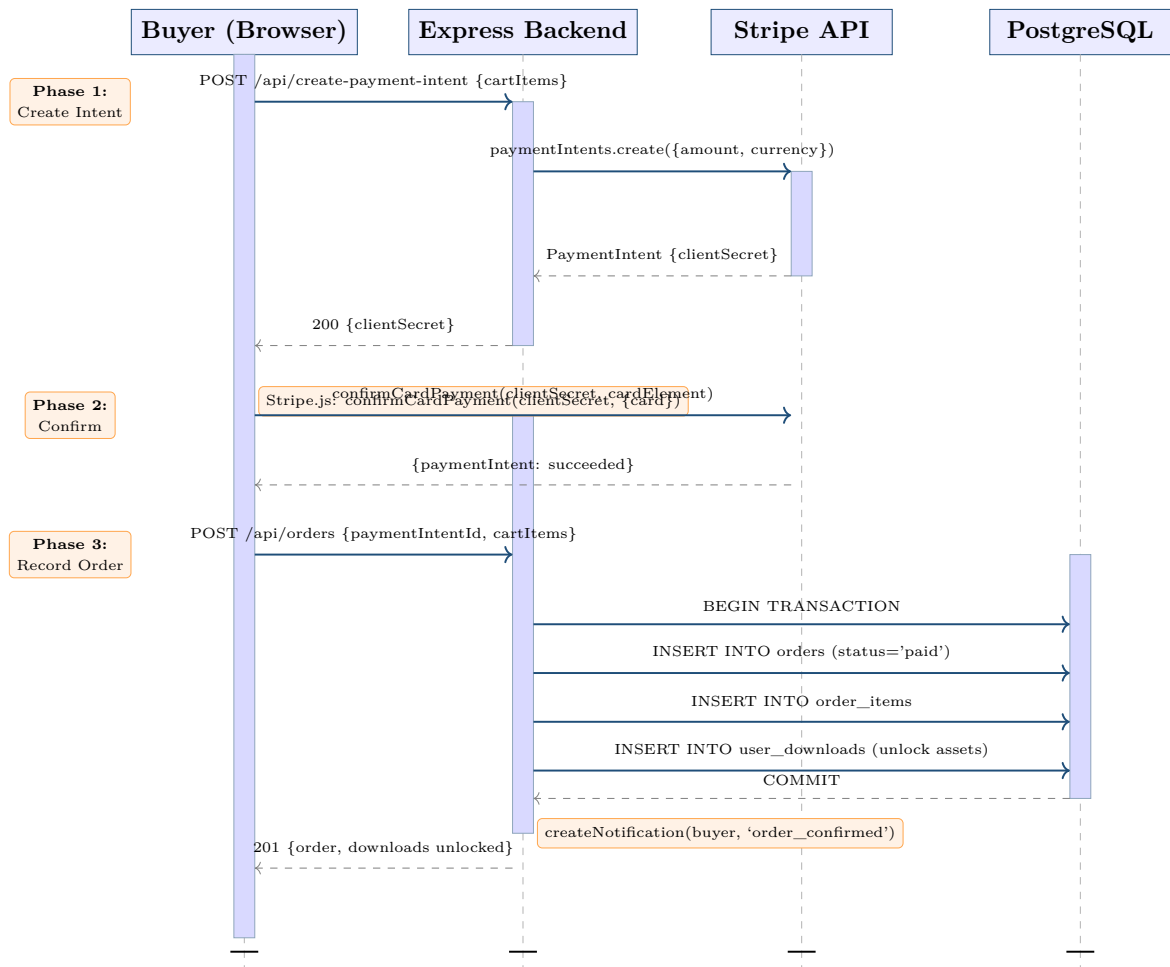


Figure 4.3: Stripe Payment Flow (Sequence Diagram)

The Stripe payment integration uses the **PaymentIntent** pattern:

1. The client sends cart contents to the backend, which creates a Stripe **PaymentIntent** for the total amount.
2. The backend returns the `clientSecret` to the frontend.
3. The frontend uses the Stripe.js SDK to confirm the card payment with the client secret — the card number never touches the backend server.
4. On successful payment confirmation, the frontend calls the orders endpoint to atomically record the order, unlock asset downloads, and send a confirmation notification.

4.2.4 Asset Viewer System

A key differentiator of IGS Marketplace is the in-browser asset preview system, which adapts the viewer component based on the asset type:

Table 4.2: Asset Categories and Corresponding Viewer Components

Category	File Formats	Viewer Component
3D Models	GLB, GLTF, FBX, OBJ	Interactive 3D viewer with orbit controls and auto-rotate
Textures	PNG, JPG, EXR	PBR map display with channel inspector
HDRIs	HDR, EXR	Panoramic 360° environment preview
Scripts	JS, PY, CS, GLSL	Syntax-highlighted code viewer
Shaders	GLSL, HLSL	Syntax-highlighted code viewer
Plugins	ZIP, DLL, unitypackage	Image gallery
Printables	STL, OBJ	3D print-ready mesh preview

4.3 Implementation

4.3.1 Asset Catalog with Filtering

The catalog endpoint supports rich query parameters for filtering by category, price range, free/paid status, and keyword search. Results are paginated and sorted by creation date. The filtering logic runs entirely server-side in PostgreSQL, ensuring efficient indexed queries on the `assets` table.

4.3.2 Interactive 3D Model Viewer

The 3D model viewer component renders GLB and GLTF assets interactively using the `@google/model-viewer` web component, which provides orbit controls, auto-rotate, shadow rendering, and AR mode. When a GLB file URL is received, the component loads the asset asynchronously and displays a thumbnail poster during loading.

4.3.3 Persistent Shopping Cart

Unlike session-based carts that disappear on logout, the IGS Marketplace cart is stored in PostgreSQL. Before adding an item, the backend checks whether the user already owns the asset through a previous purchase — if so, the addition is rejected with

an informative error. This ownership check prevents duplicate purchases and applies equally to free asset claims.

4.3.4 File Storage via Supabase

All asset files (3D models, textures, thumbnails) are stored in Supabase Storage via the `storageService` module. The service uploads the binary buffer to the appropriate bucket and returns a permanent CDN-backed public URL. Public URLs are stored directly in the `assets` table, eliminating the need for signed URLs on every download request for publicly approved assets.

4.4 Testing

Table 4.3: Sprint 2 — Test Cases

TC	Test Case	Expected Result	Status
TC-09	Browse catalog (unauthenticated)	Returns approved assets list	PASS
TC-10	Filter by category	Returns only assets in that category	PASS
TC-11	Filter by price range	Returns assets within price range	PASS
TC-12	Seller uploads an asset	File uploaded to Supabase, asset created as <i>pending</i>	PASS
TC-13	Add to cart (not owned)	Cart item created	PASS
TC-14	Add to cart (already owned)	400 error returned	PASS
TC-15	Stripe payment (test card)	PaymentIntent confirmed, order created	PASS
TC-16	Download asset (owned)	Download URL returned	PASS
TC-17	Download asset (not owned)	403 Forbidden	PASS
TC-18	Claim free asset	<code>user_downloads</code> record created	PASS
TC-19	View GLB in 3D viewer	Model renders correctly in browser	PASS

4.5 Sprint Review

4.5.1 Deliverables

- Full asset catalog with filtering by category, price, free/paid status, and keyword search.
- Interactive 3D viewer for GLB/GLTF assets; specialized viewers for textures, HDRIs, and code files.
- Seller upload workflow with Supabase file storage and two-phase submission.
- Multiple file format support per asset.
- Persistent shopping cart stored in PostgreSQL.
- Stripe PaymentIntent integration with test mode validation.
- Order management with status tracking and download unlocking.
- User library page showing all purchased and downloaded assets.

4.5.2 Sprint Screenshots

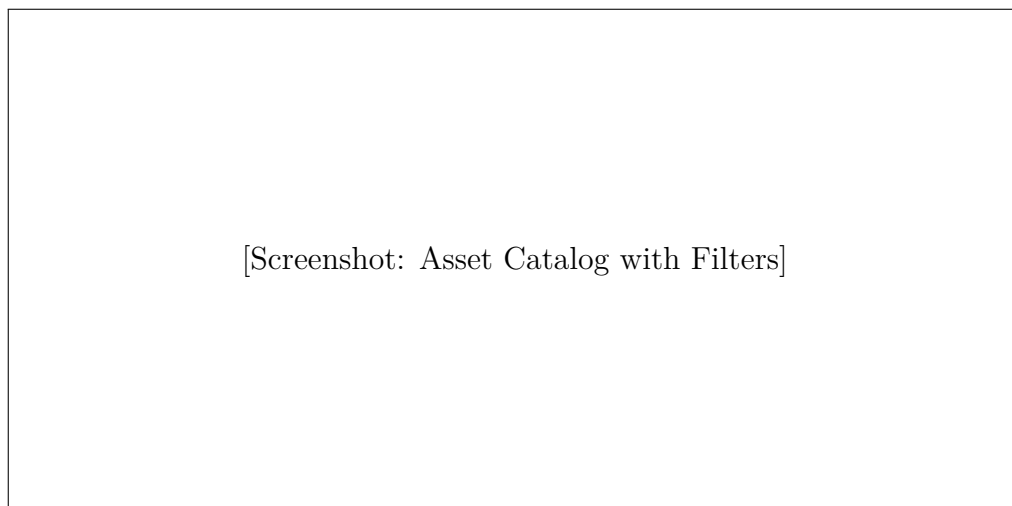


Figure 4.4: Asset Catalog with Category and Price Filters

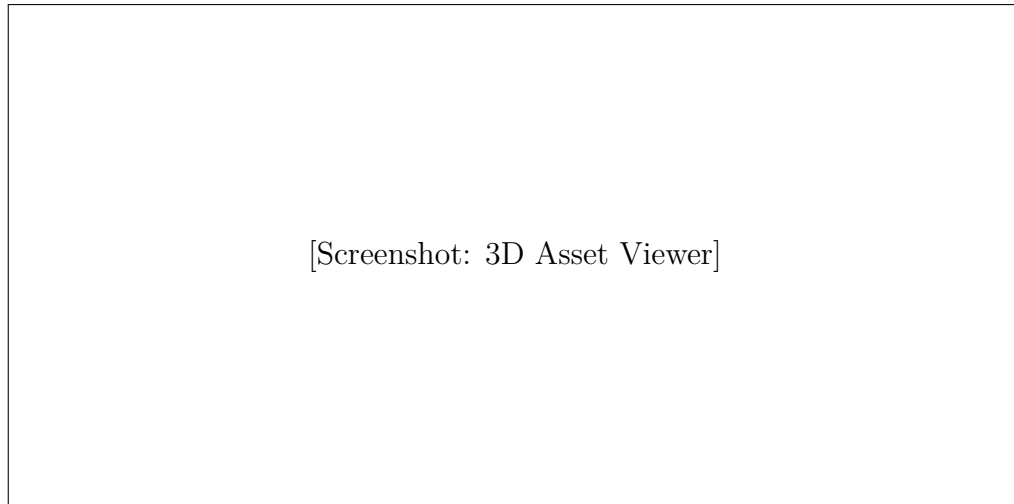


Figure 4.5: Interactive 3D Model Viewer

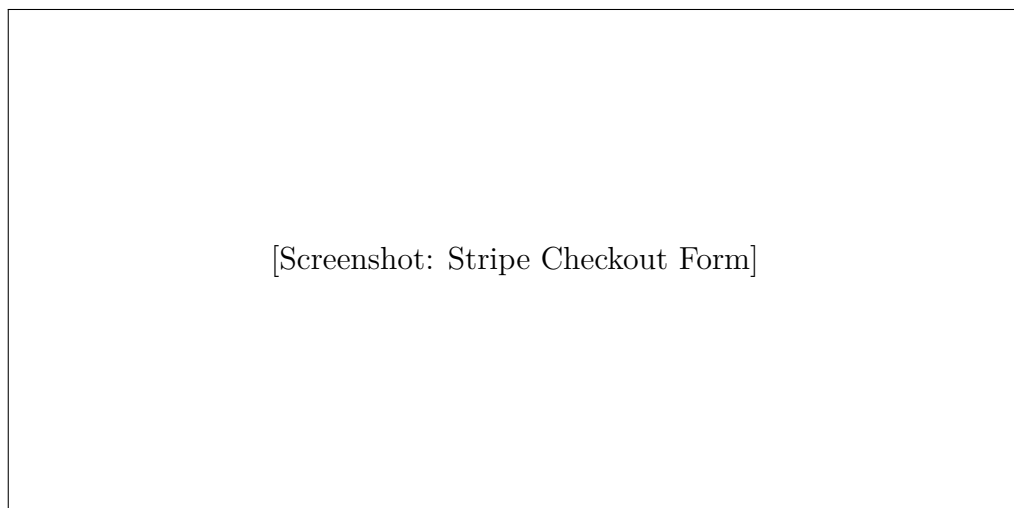


Figure 4.6: Stripe Payment Checkout Form

Conclusion

Sprint 2 successfully delivered the core marketplace functionality. Users can now discover, preview, and purchase digital assets through a complete e-commerce flow. The 3D viewer and secure Stripe integration distinguish IGS Marketplace from simpler file-sharing solutions. Sprint 3 will add content quality features, administrative tools, and subscription plans.

Chapter 5

Sprint 3: Content Management and Administration

Introduction

Sprint 3 focuses on the trust and governance layer of the marketplace. It introduces the review and rating system, an in-app notification center, subscription plans, and a comprehensive admin moderation panel. These features ensure content quality, user engagement, and platform sustainability.

5.1 Sprint Backlog

Sprint 3 — Duration: 3 weeks

Goal: Deliver content moderation tools, a review system, notifications, and subscription management.

Table 5.1: Sprint 3 — User Stories

ID	User Story	Priority	Points
US-16	As a buyer, I want to submit a review and rating for an asset I own, so that other users can benefit from my feedback.	High	5
US-17	As any user, I want to like an asset so that I can save favorites and show appreciation.	Medium	2
US-18	As an admin, I want to approve or reject pending assets so that only quality content is published.	High	5
US-19	As a user, I want to receive in-app notifications for key events so that I stay informed.	High	8
US-20	As an admin, I want to view platform analytics (total sales, top assets) so that I can make data-driven decisions.	High	5
US-21	As a user, I want to subscribe to a plan so that I gain additional benefits (download quotas, features).	Medium	5
US-22	As an admin, I want all moderation actions logged in an audit trail.	Medium	2
US-23	As a seller, I want to receive a notification when my asset is approved or rejected.	High	3
Total Story Points:			35

5.2 Analysis and Design

5.2.1 Database Schema — Reviews, Notifications, Subscriptions

Three new migration groups extend the database in Sprint 3:

Reviews & Likes:

- **reviews:** Stores user reviews with `rating` (1–5), `comment`, an AI-generated `ai_summary` field, and a `flagged_for_moderation` boolean.
- **asset_likes:** Junction table tracking which users liked which assets, with a composite primary key.

Notifications:

- **notifications:** Stores in-app notifications with `type`, `title`, `message`, `is_read`, and a `related_id` reference for deep-linking.

Subscriptions:

- **subscription_plans:** Defines available plans (Free, Subscriber, Seller) with monthly price, download limits, and a JSONB `features` column.
- **user_subscriptions:** Tracks the active subscription per user with start/end dates and an `is_active` flag.

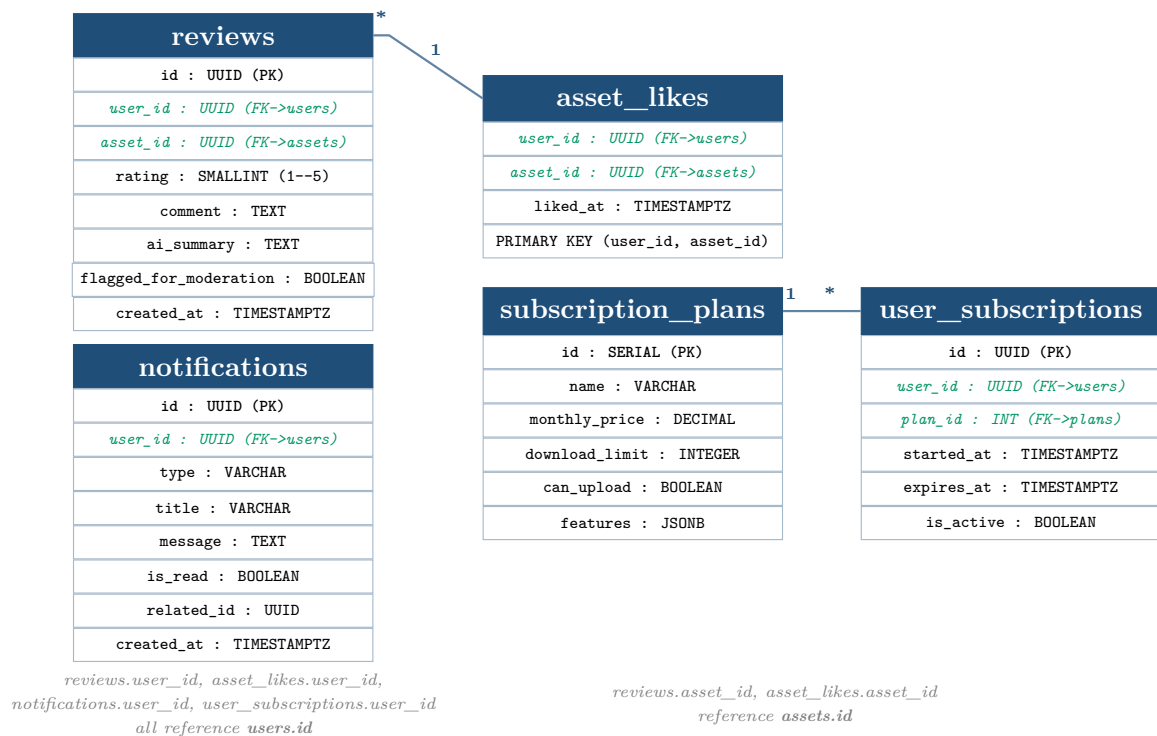


Figure 5.1: Sprint 3 — Reviews, Notifications and Subscriptions ERD

5.2.2 Admin Moderation Use Case

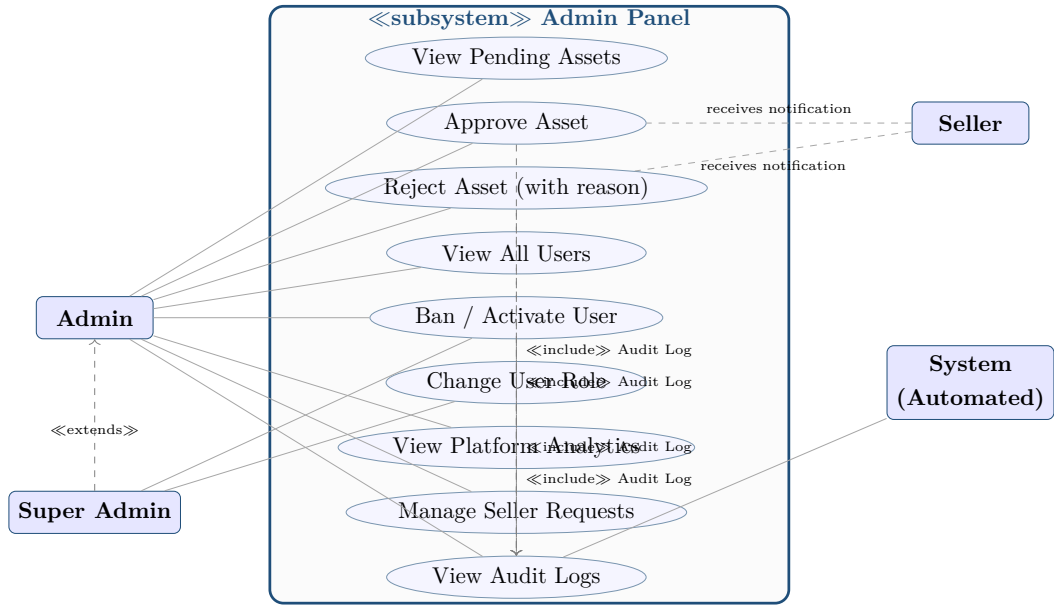


Figure 5.2: Admin Moderation Use Case Diagram

5.2.3 Subscription Plans

Table 5.2: Subscription Plan Tiers

Plan	Monthly Price	Download Limit	Key Benefits
Free	0 TND	5 free assets/month	Browse, buy, download free assets
Subscriber	9.99 TND	20 free assets/month	Priority support, discounts
Seller	19.99 TND	Unlimited	Upload assets, seller dashboard

5.3 Implementation

5.3.1 Review and Rating System

Reviews are restricted to users who have purchased or downloaded an asset. The back-end enforces this ownership check before accepting any review submission. Once a review is created, the asset’s `rating_average` and `rating_count` fields are automati-

cally recalculated via an aggregate query, ensuring the displayed rating always reflects the current state.

5.3.2 Notification System

The notification system follows a **pull-based** design: the backend creates notification records in the `notifications` table whenever a significant event occurs (asset approved, order confirmed, new review received, role changed). The frontend fetches notifications on page load and on user interactions, displaying an unread count badge in the navigation bar. Users can mark individual notifications as read or clear all at once.

Notification types include: `asset_approved`, `asset_rejected`, `order_confirmed`, `new_review`, `seller_request_approved`, `role_changed`, and more.

5.3.3 Admin Moderation Panel

The admin moderation panel provides a paginated queue of pending asset submissions. For each submission, the admin can:

- **Preview** the asset files and metadata before making a decision.
- **Approve** the asset, setting its status to `approved` and making it visible in the public catalog.
- **Reject** the asset with a written reason, which is forwarded to the seller via a notification.

All moderation actions are recorded in the audit log with the moderator's user ID, the action performed, and a timestamp.

5.3.4 Platform Analytics Dashboard

The admin analytics endpoint aggregates sales data across the entire platform, returning: total revenue, total orders, total assets sold, top-selling assets by revenue, and a time-series revenue chart. This data feeds the admin dashboard cards and charts.

5.4 Testing

Table 5.3: Sprint 3 — Test Cases

TC	Test Case	Expected Result	Status
TC-20	Submit review (asset owned)	Review created, asset rating recalculated	PASS
TC-21	Submit review (asset not owned)	403 Forbidden	PASS
TC-22	Like an asset	Like record created, count incremented	PASS
TC-23	Admin approves asset	Status set to approved, seller notified	PASS
TC-24	Admin rejects asset with reason	Status set to rejected, reason forwarded	PASS
TC-25	Fetch unread notifications	Returns all unread notifications for user	PASS
TC-26	Mark notification as read	<code>is_read</code> set to true	PASS
TC-27	Upgrade to Seller plan	<code>user_subscriptions</code> record updated	PASS
TC-28	Moderation action performed	Audit log entry created	PASS

5.5 Sprint Review

5.5.1 Deliverables

- Review and rating system with ownership enforcement and automatic rating recalculation.
- Asset like/favorite system with per-asset counter.
- In-app notification center with unread badge and mark-as-read functionality.
- Admin moderation panel: view pending assets, approve or reject with written reason.
- Platform analytics dashboard (total sales, top assets, revenue overview).

- Three-tier subscription system (Free, Subscriber, Seller) with plan management UI.
- Full audit trail for all admin moderation actions.

5.5.2 Sprint Screenshots

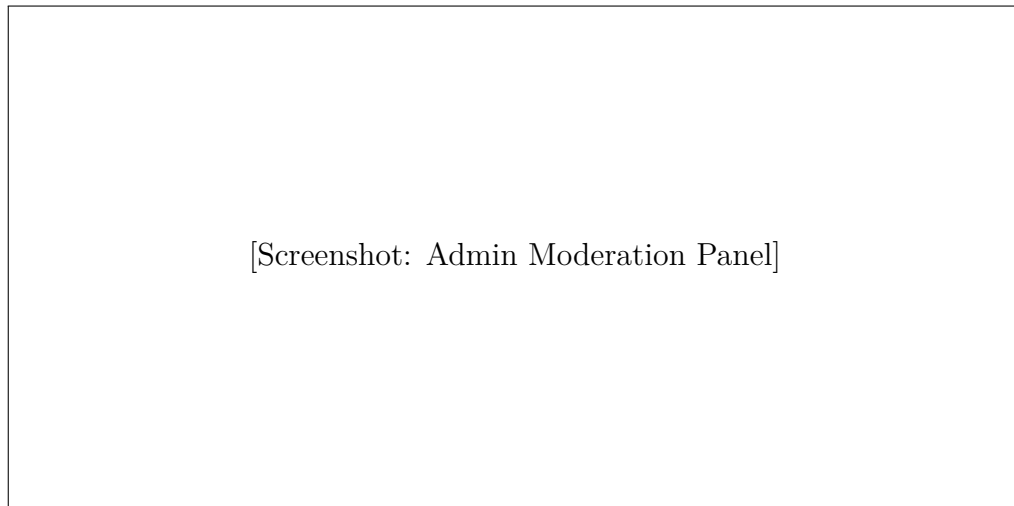


Figure 5.3: Admin Moderation Panel — Pending Assets Queue

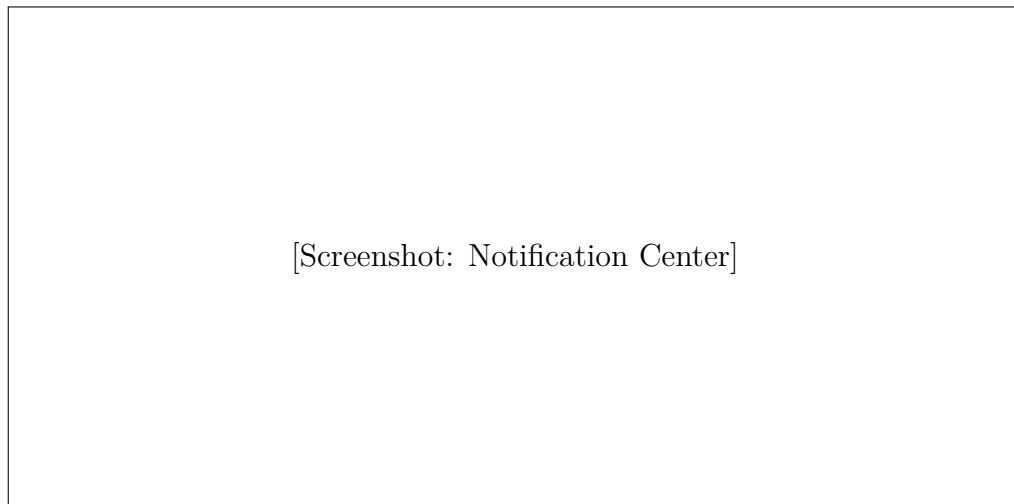


Figure 5.4: Notification Center with Unread Badge

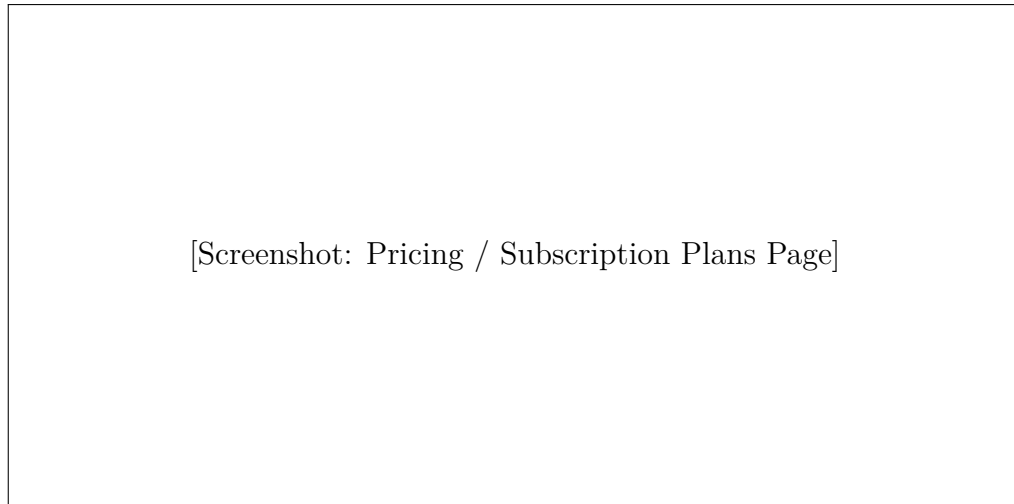


Figure 5.5: Subscription Plans (Pricing Page)

Conclusion

Sprint 3 has significantly enriched the platform with trust-building and governance features. The combination of user reviews, admin moderation, and subscription tiers creates a sustainable ecosystem. Sprint 4 will introduce role-specific advanced features including the intern workflow, seller analytics, and community features.

Chapter 6

Sprint 4: Role-Based Workflows and Community Features

Introduction

Sprint 4 extends the platform with features tailored to specific user roles: a structured intern access workflow, seller analytics, asset edit request management, peer-to-peer asset trading, and community group functionality. This sprint makes the RBAC system come to life in practical, day-to-day operational terms.

6.1 Sprint Backlog

Sprint 4 — Duration: 3 weeks

Goal: Deliver role-specific advanced features for interns, sellers, and the broader community.

Table 6.1: Sprint 4 — User Stories

ID	User Story	Priority	Points
US-24	As an intern, I want to request access to a paid asset so that I can use it for IGS projects without payment.	High	5
US-25	As an admin, I want to approve or reject intern asset requests and optionally generate vouchers.	High	5
US-26	As an intern, I want to redeem a voucher code to unlock an asset.	High	3
US-27	As a seller, I want to view my sales analytics (views, downloads, revenue) so that I can track performance.	High	5
US-28	As a seller, I want to submit an edit request for an approved asset so that I can update its content.	Medium	5
US-29	As a user, I want to propose a trade with another user (my asset for theirs) so that we can exchange assets.	Medium	8
US-30	As a user, I want to create and join community groups so that I can connect with others around shared interests.	Medium	5
US-31	As an external user, I want to request the seller role so that I can publish my own assets.	High	3
Total Story Points:			39

6.2 Analysis and Design

6.2.1 Database Schema — Intern and Advanced Features

Sprint 4 introduces several new database entities:

Intern Features:

- **intern_asset_requests:** Tracks intern requests for paid assets, with status (pending, approved, rejected) and an optional generated voucher code.
- **intern_vouchers:** Stores single-use voucher codes generated by admins for approved intern requests.

Asset Edit Requests:

- **asset_edit_requests**: Allows sellers to propose changes to approved assets (title, description, price, file update) without requiring a full re-upload and re-moderation cycle. Proposed changes are stored as JSONB.

Community and Trading:

- **trades**: Peer-to-peer trade offers between users, linking an offered asset to a requested asset, with a negotiation status.
- **groups** and **group_members**: Community groups with member roles (admin, member) and optional privacy settings.

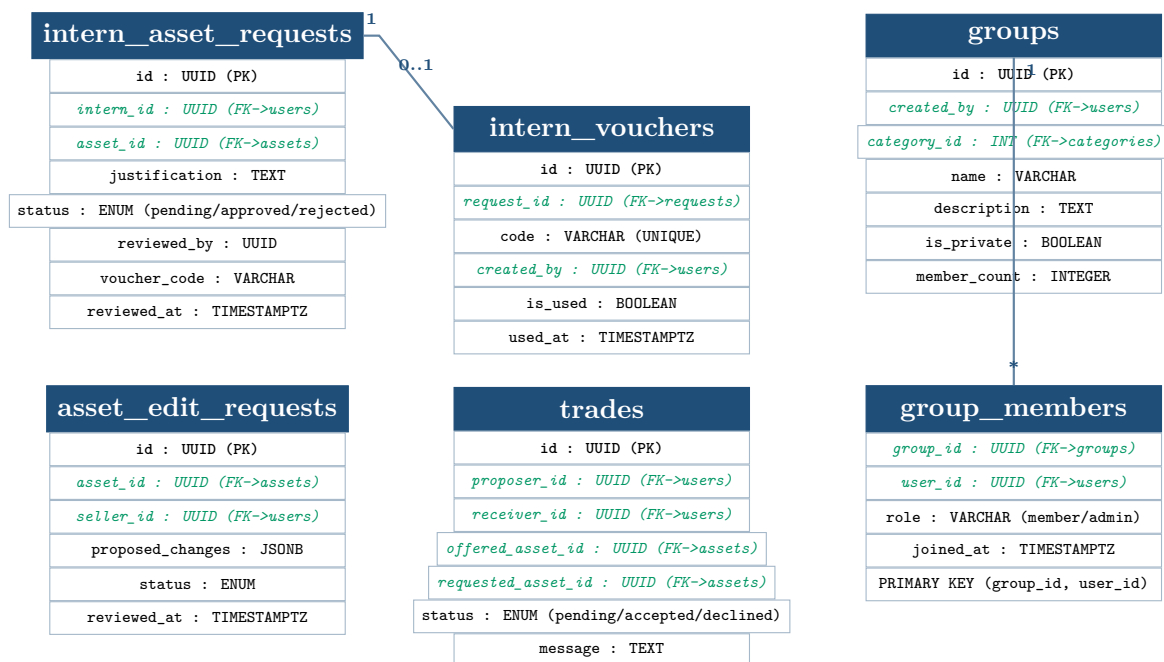


Figure 6.1: Sprint 4 — Entity-Relationship Diagram

6.2.2 Intern Asset Request Workflow

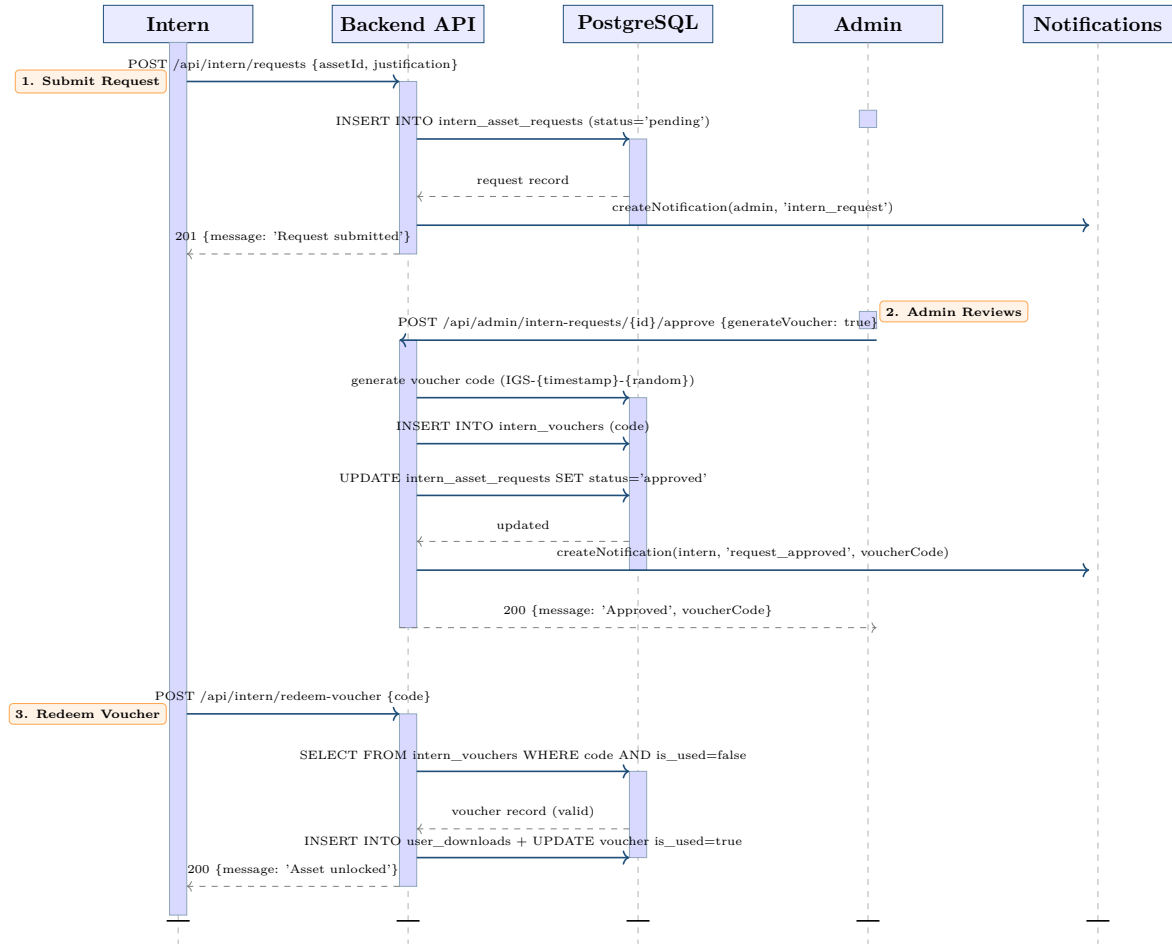


Figure 6.2: Intern Asset Request Workflow (Sequence Diagram)

The intern asset request workflow proceeds through the following steps:

1. The intern browses the catalog, finds a paid asset needed for an IGS project, and submits a request with a written justification.
2. The admin reviews the request in the moderation panel.
3. If approved, the admin optionally generates a single-use voucher code.
4. The intern receives a notification with the voucher code (if issued).
5. The intern redeems the voucher, which unlocks the asset in their personal library.

6.2.3 Peer-to-Peer Trading System

The trading system allows two users who each own different assets to propose an exchange without monetary transaction:

1. User A submits a trade proposal: *"I offer asset X in exchange for asset Y."* The system verifies User A owns asset X.
2. User B (owner of Y) is notified and can accept or decline the trade.
3. If accepted, both assets are atomically added to each other's libraries and the trade record is marked **completed**.

6.2.4 Seller Analytics

The seller analytics dashboard aggregates key performance metrics for each seller:

- **Total revenue:** Sum of all completed order items for the seller's assets.
- **Total orders:** Count of distinct orders containing the seller's assets.
- **Total downloads:** Count of unique `user_downloads` records for the seller's assets.
- **Active assets:** Count of currently approved assets.
- **Per-asset breakdown:** Individual revenue, download count, view count, and average rating per asset.

6.3 Implementation

6.3.1 Intern Asset Request Controller

When an admin approves an intern request and opts to generate a voucher, the system creates a unique, single-use code in the `intern_vouchers` table, updates the request status to **approved**, and dispatches a notification to the intern containing either the voucher code or a direct unlock confirmation. If the admin approves without a voucher, the asset is added directly to the intern's library.

6.3.2 Seller Analytics

The analytics endpoint uses a single aggregate SQL query joining `assets`, `order_items`, and `user_downloads` tables, filtered by the authenticated seller's user ID. This provides a real-time, consistent view of all sales performance metrics. The frontend renders the data using summary cards and a per-asset breakdown table.

6.3.3 Asset Edit Requests

Sellers can submit edit requests for assets that are already published (`approved` status). The proposed changes are stored as a JSONB object in the `asset_edit_requests` table, pending admin review. This allows the platform to maintain editorial control over changes to live assets without requiring a full re-submission and review cycle.

6.3.4 Community Groups

Groups are created by any registered user and support optional privacy settings. The group creator is automatically added as the group admin. Members can be invited or can request to join public groups. Groups are organized by asset category (e.g., a group for 3D modelers, one for texture artists), enabling targeted discussions.

6.4 Testing

Table 6.2: Sprint 4 — Test Cases

TC	Test Case	Expected Result	Status
TC-29	Intern requests paid asset	Request record created as <i>pending</i>	PASS
TC-30	Admin approves with voucher	Voucher generated, intern notified	PASS
TC-31	Intern redeems valid voucher	Asset added to library, voucher invalidated	PASS
TC-32	Intern redeems invalid voucher	400 error returned	PASS
TC-33	Seller views analytics	Revenue, downloads, and orders returned	PASS
TC-34	Seller submits edit request	Edit request created as pending	PASS
TC-35	User proposes trade	Trade record created, counterpart notified	PASS
TC-36	User accepts trade	Both assets swapped in libraries	PASS
TC-37	Create community group	Group created, creator is group admin	PASS
TC-38	External user requests seller role	Seller request submitted for admin review	PASS

6.5 Sprint Review

6.5.1 Deliverables

- Complete intern asset request workflow with voucher generation and redemption.
- Seller analytics dashboard with revenue, download count, and per-asset breakdown.
- Asset edit request system allowing sellers to propose post-approval changes.
- Peer-to-peer asset trading system with accept/decline flow and atomic ownership transfer.

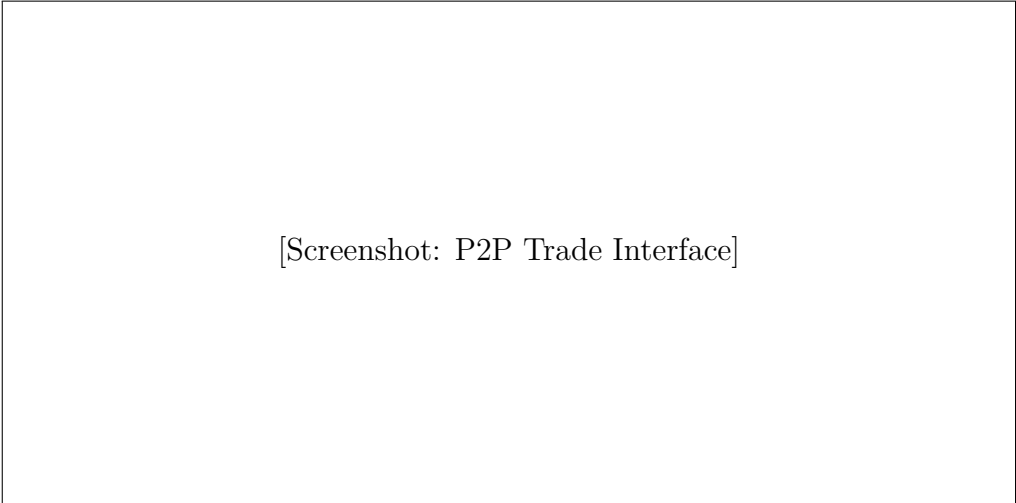
- Community groups with member management and group-admin role.
- External user seller role request flow with admin approval/rejection.

6.5.2 Sprint Screenshots



[Screenshot: Intern Asset Request and Voucher Redemption]

Figure 6.3: Intern Asset Request and Voucher Redemption Page



[Screenshot: P2P Trade Interface]

Figure 6.4: Peer-to-Peer Asset Trade Interface

Conclusion

Sprint 4 has delivered the role-specific advanced features that make the RBAC system operationally meaningful. The intern workflow, seller analytics, and trading system add significant value for each user segment. Sprint 5 will develop the companion internal workspace application for IGS team collaboration.

Chapter 7

Sprint 5: Internal Collaboration Platform

Introduction

Sprint 5 introduces the **IGS Workspace**, a companion internal web application designed exclusively for IGS employees and interns. This dedicated React application provides a ClickUp-inspired dashboard with team management, task tracking, project/ERP management, and a real-time chat system. Both applications share the same backend API and JWT authentication, enabling seamless Single Sign-On.

7.1 Sprint Backlog

Sprint 5 — Duration: 4 weeks

Goal: Deliver a complete internal team management and collaboration platform integrated with the marketplace backend.

Table 7.1: Sprint 5 — User Stories

ID	User Story	Priority	Points
US-32	As an internal user, I want a dedicated workspace dashboard so that I can manage my work without mixing it with the public marketplace.	High	5
US-33	As an admin/employee, I want to manage team members (add, assign positions, set supervisors).	High	8
US-34	As any internal user, I want to view and manage tasks assigned to me with priority and status.	High	5
US-35	As an admin, I want to create and manage projects with milestones and team assignments.	High	8
US-36	As any internal user, I want to send and receive real-time messages in team channels.	High	8
US-37	As any internal user, I want to send direct messages (DMs) to colleagues.	Medium	5
US-38	As a user, I want to add emoji reactions to messages.	Low	3
US-39	As a user, I want to navigate the workspace with role-adapted menus (interns see fewer sections).	Medium	3
Total Story Points:			45

7.2 Analysis and Design

7.2.1 IGS Workspace Architecture

The workspace is a separate React + Vite application (port 5174) that reads the shared JWT from `localStorage` (set during marketplace login), communicates with the same Express backend on port 3001, and enforces an internal-user guard at the layout level. Any unauthenticated or external user attempting to access the workspace URL is immediately redirected.

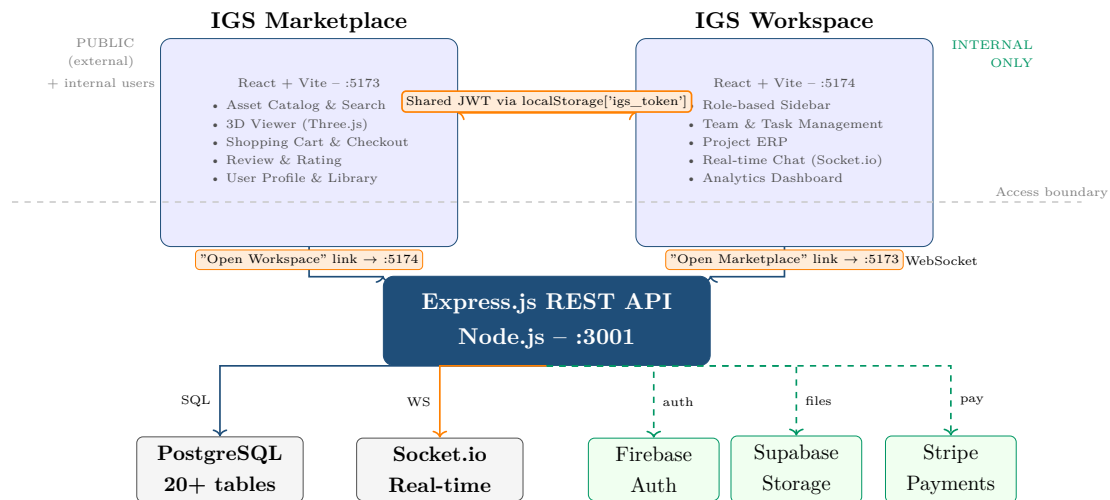


Figure 7.1: IGS Workspace Architecture and Integration with Marketplace

7.2.2 Database Schema — Workspace and Chat

Sprint 5 introduces six new database migration groups:

- **Workspace Tasks:** The `workspace_tasks` table stores tasks with title, description, status (`todo` / `in_progress` / `done`), priority (`low` / `medium` / `high` / `urgent`), assignee, and due date.
- **Projects:** The `projects` and `project_members` tables support project definition with milestones, start/end dates, and team assignment.
- **Chat Channels:** `chat_channels` and `channel_members` define named public or private channels within the workspace.
- **Messages:** `chat_messages` stores individual channel messages with optional `parent_id` for threaded replies; `message_reactions` stores emoji reactions with user attribution.
- **Direct Messages:** `direct_messages` handles private 1-on-1 conversations between two users.
- **Team Structure:** `company_positions` and `team_supervisors` define the organizational hierarchy, mapping interns and juniors to their supervisors.

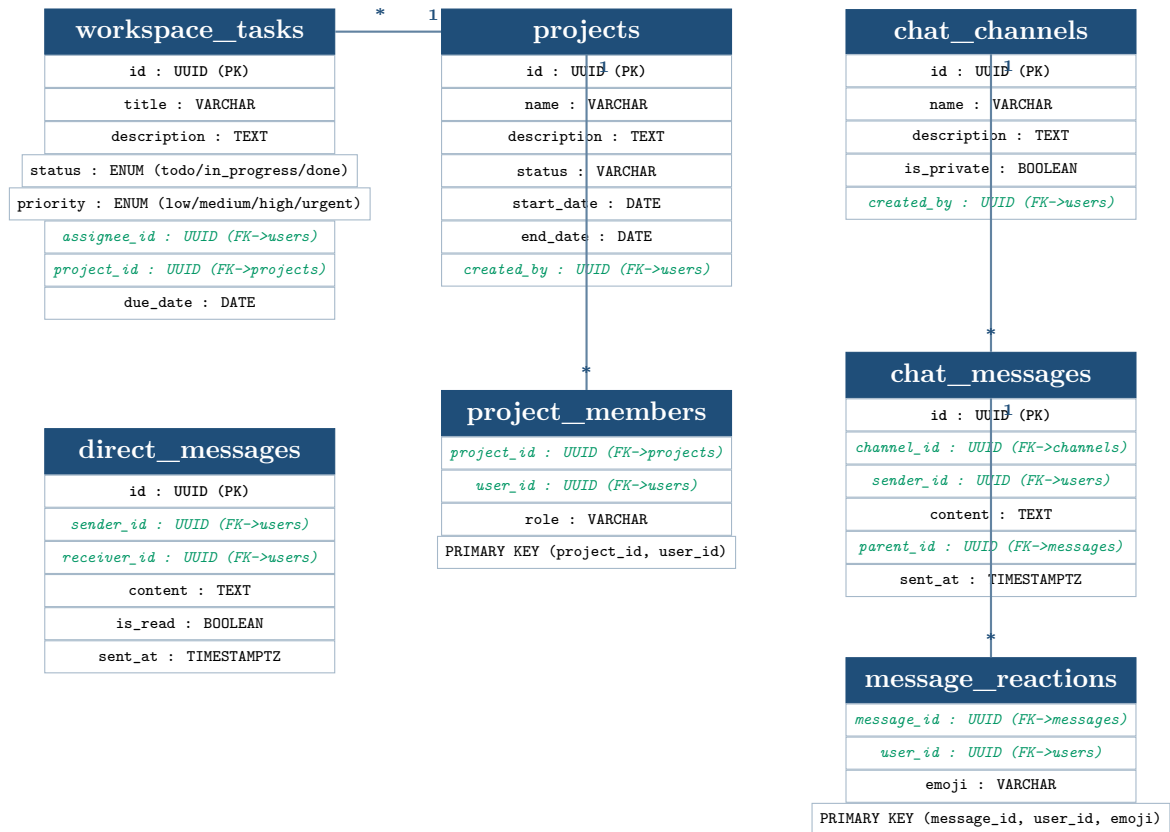


Figure 7.2: Sprint 5 — Workspace and Chat Entity-Relationship Diagram

7.2.3 Role-Based Navigation

The workspace sidebar adapts its navigation sections based on the authenticated user's role level:

Table 7.2: Workspace Sidebar Navigation by Role

Section	Min. Role	Pages
General	intern	Dashboard, My Tasks, Team Directory, Calendar
Analytics	employee	Reports, Performance Charts
Management	admin	Asset Management, User Management, Orders
System	super_admin	Roles & Permissions, System Settings

7.2.4 Real-Time Chat Architecture (Socket.io)

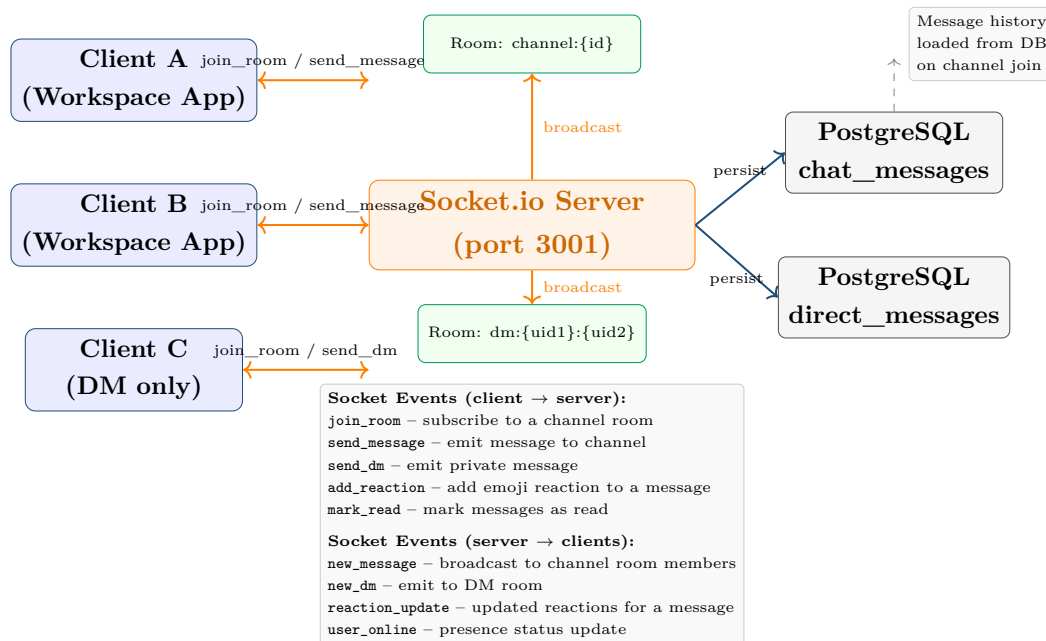


Figure 7.3: Real-Time Chat Architecture with Socket.io

The chat system uses Socket.io rooms for efficient message broadcasting:

- Each channel maps to a Socket.io room named `channel:{channelId}`.
- Each DM pair shares a deterministic room named `dm:{userId1}:{userId2}` (IDs ordered alphabetically to avoid duplicates).
- When a message is sent, the server persists it to PostgreSQL and broadcasts it to all room members in real time.
- Full message history is loaded from the database when a user joins a channel.

7.3 Implementation

7.3.1 Workspace Layout and Auth Guard

The `WorkspaceLayout` component acts as the entry gate for the entire workspace application. It reads the shared JWT from `localStorage`, verifies the user is authenticated and of `internal` type, and renders the sidebar/topbar shell. Any violation of these conditions results in an immediate redirect to the public marketplace login page or an unauthorized page.

7.3.2 Real-Time Chat System

The Socket.io server is initialized alongside the Express HTTP server. It handles four primary events:

- **join_room:** Subscribes the socket to a channel or DM room and loads recent message history.
- **send_message:** Persists the message to the database and broadcasts it to all room members.
- **send_dm:** Sends a private message to the DM room of the two participants.
- **add_reaction:** Appends an emoji reaction to a message and broadcasts the updated reaction set.

7.3.3 Task Management

Tasks follow a Kanban-inspired status lifecycle: `todo` → `in_progress` → `done`. Each task can be assigned to one team member, tagged with a priority level, and associated with a project. Filtering by status, priority, assignee, and project is supported both on the backend and the frontend.

7.3.4 Workspace Dashboard

The main dashboard provides internal users with an at-a-glance view of:

- **Stats cards:** Total active tasks, team size, current projects, and tasks completed this week.
- **My Tasks widget:** Lists the current user's assigned tasks ordered by priority and due date.
- **Activity feed:** Recent platform events (asset approvals, new orders, team updates).
- **Quick actions:** One-click shortcuts to create a task, open chat, or view reports.

7.4 Testing

Table 7.3: Sprint 5 — Test Cases

TC	Test Case	Expected Result	Status
TC-39	External user accesses workspace URL	Redirected to unauthorized page	PASS
TC-40	Intern logs in and accesses workspace	Dashboard shown with limited navigation	PASS
TC-41	Admin sees full navigation	All 4 sidebar sections visible	PASS
TC-42	Create task with assignee and due date	Task created, assignee notified	PASS
TC-43	Move task to in_progress	Status updated in real time	PASS
TC-44	Send message in channel	Message broadcast to all channel members	PASS
TC-45	Send DM to colleague	Private message delivered instantly	PASS
TC-46	Add emoji reaction to message	Reaction saved and broadcast	PASS
TC-47	Create project with team members	Project created, members notified	PASS
TC-48	SSO between marketplace and workspace	Shared JWT accepted by both applications	PASS

7.5 Sprint Review

7.5.1 Deliverables

- IGS Workspace application (React + Vite, port 5174) with dark sidebar and topbar layout.
- Role-based sidebar navigation adapting to the authenticated user's internal role.
- Team directory with position management and supervisor assignment.
- Task management system with priority, status (Kanban flow), assignee, and due date.

- Project management (ERP) with milestone tracking and team assignment.
- Real-time chat with channels, direct messages, emoji reactions, and read receipts.
- Seamless Single Sign-On between the marketplace (port 5173) and workspace (port 5174) via shared JWT.
- Bidirectional navigation links between both applications.

7.5.2 Sprint Screenshots



Figure 7.4: IGS Workspace — Main Dashboard

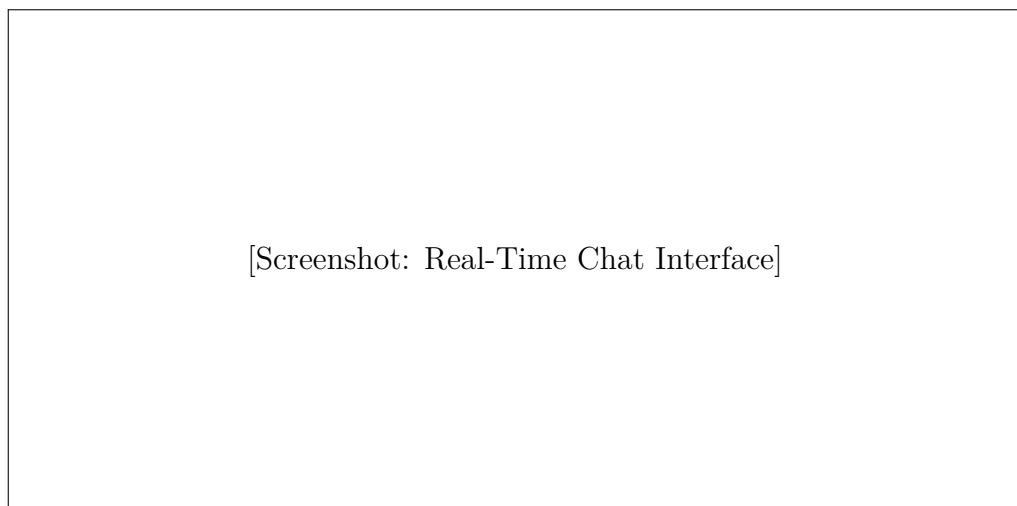


Figure 7.5: IGS Workspace — Real-Time Team Chat with Emoji Reactions



Figure 7.6: IGS Workspace — Task Management

Conclusion

Sprint 5 successfully delivered the IGS Workspace, transforming the project from a standalone marketplace into a comprehensive platform ecosystem. The combination of task management, project tracking, and real-time communication within a single internally-facing application significantly improves the studio's operational efficiency. Sprint 6 will add artificial intelligence capabilities to the platform.

Chapter 8

Sprint 6: Artificial Intelligence Integration

Introduction

Sprint 6 integrates Artificial Intelligence capabilities into the IGS Marketplace platform using the **Anthropic AI API**, a state-of-the-art large language model (LLM) service. This sprint introduces intelligent review analysis, automated sentiment classification for content moderation, and the initial implementation of a conversational assistant chatbot. AI features represent a significant differentiator for the platform, moving it from a static catalog toward an intelligent, self-improving marketplace.

8.1 Sprint Backlog

Sprint 6 — Duration: 3 weeks

Goal: Integrate AI-powered review analysis, sentiment-based moderation assistance, and a marketplace assistant chatbot.

Table 8.1: Sprint 6 — User Stories and Status

ID	User Story	Priority	Status
US-40	As a seller, I want to receive an AI-generated sentiment summary of my asset’s reviews so that I can quickly understand customer feedback.	High	Done
US-41	As an admin, I want flagged reviews (negative sentiment) to be automatically highlighted so that I can moderate efficiently.	High	Done
US-42	As a user, I want a chatbot assistant that can answer my questions about assets and the platform.	High	In Progress
US-43	As a seller, I want the chatbot to suggest relevant tags for my asset based on the description I provide.	Medium	Planned
US-44	As a user, I want the platform to suggest similar assets based on the one I am viewing.	Medium	Planned

8.2 AI System Architecture

8.2.1 Design Principles

The AI integration follows a **server-side processing** architecture: the React frontend never calls the AI API directly. All AI requests are routed through the Express backend, which calls the Anthropic API and returns structured results to the client. This architecture ensures:

- **Security:** The API key is stored as a server-side environment variable and never exposed to the browser.
- **Control:** Rate limiting, caching, and prompt engineering are managed centrally on the backend.
- **Portability:** The AI provider can be replaced or upgraded without any frontend changes.

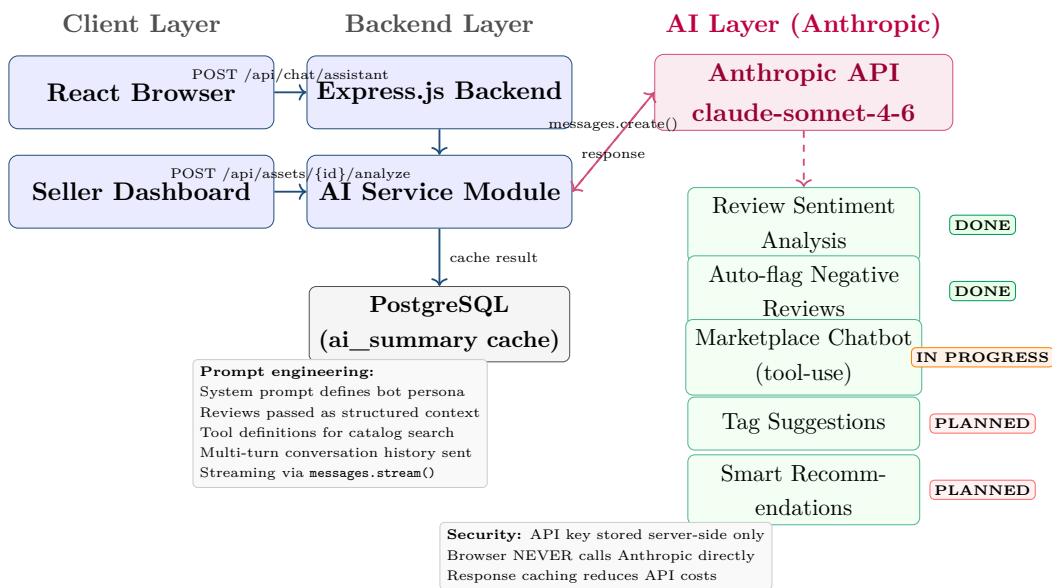


Figure 8.1: AI Integration Architecture — Server-side LLM API calls

8.2.2 AI Model Specification

Table 8.2: AI Model and API Specification

Parameter	Value
AI Provider	Anthropic
Model Family	Haiku (fast, efficient, low-cost)
Model Identifier	claude-haiku-4-5-20251001
Max Tokens (analysis)	1024
Max Tokens (chatbot)	512
Response Format	Structured JSON (for analysis) / Plain text (for chatbot)
API Interface	Anthropic Messages API (POST /v1/messages)
Prompt Style	Single-turn (analysis) / Multi-turn with history (chatbot)
Authentication	Server-side API key via <code>ANTHROPIC_API_KEY</code> env var

The **Haiku** model was selected for its optimal balance of intelligence and latency at minimal cost — it processes thousands of review analysis requests per day without significant API spend, while still producing coherent, structured outputs suitable for a production marketplace.

8.3 AI Review Analysis (Completed)

8.3.1 Feature Description

The review analysis feature is accessible from the seller dashboard as an “Analyze with AI” action on any asset with at least one review. When triggered, the backend:

1. Fetches all reviews for the asset from PostgreSQL (rating + comment text).
2. Constructs a structured prompt containing the reviews and requesting a JSON-formatted analysis.
3. Sends the prompt to the Anthropic AI API.
4. Parses the structured JSON response returned by the model.
5. Returns the parsed analysis to the frontend and optionally caches the result in the `ai_summary` column of the `reviews` table.

8.3.2 Analysis Output Structure

The AI model returns a structured JSON object containing:

- **sentiment:** Overall sentiment classification — `positive`, `neutral`, or `negative`.
- **overall_summary:** A 2–3 sentence plain-language synthesis of all reviews.
- **key_positives:** An array of the most frequently praised aspects.
- **key_negatives:** An array of the most commonly criticized aspects.
- **improvement_suggestions:** Actionable recommendations for the seller.

The seller dashboard renders this output as a structured card with color-coded sentiment badge, summary text, and expandable bullet lists for positives, negatives, and suggestions.

8.3.3 Automated Negative Review Flagging

Reviews submitted with a rating of 1 or 2 stars are automatically analyzed at creation time. If the AI model classifies the review’s sentiment as **negative**, the review is flagged for admin moderation (`flagged_for_moderation = true`). This creates an automated moderation queue that surfaces potentially harmful or abusive reviews for human review without requiring admins to manually read every low-rating submission.

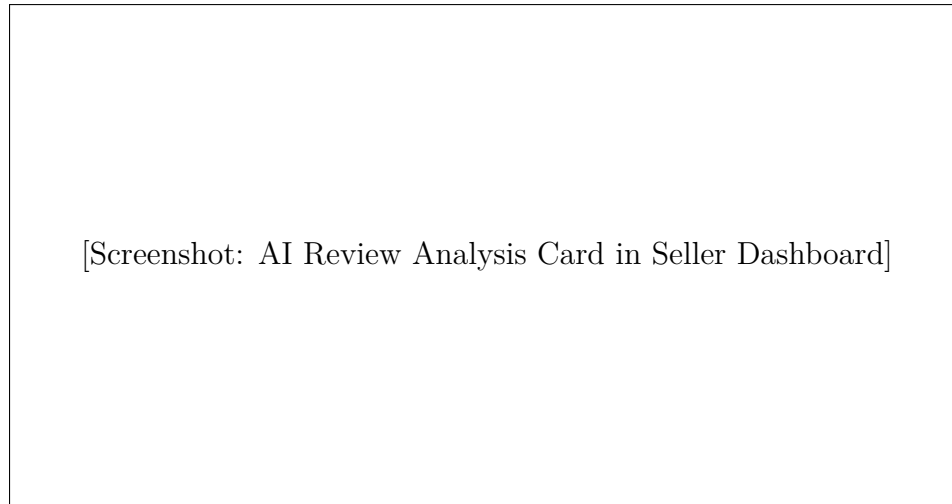


Figure 8.2: AI-Powered Review Analysis — Seller Dashboard

8.4 Marketplace Assistant Chatbot (In Progress)

8.4.1 Design

The marketplace assistant chatbot is designed as a **context-aware conversational agent** available to all users, including unauthenticated visitors. Its design principles:

- A detailed **system prompt** defines the assistant’s persona and domain knowledge: all 13 asset categories, available file formats, the seller programme, subscription plans, the trading system, and platform policies.
- The assistant is **context-aware**: when a user is authenticated, their role (guest / member / seller / staff) is injected into the system prompt to personalize responses.
- The frontend sends the **full conversation history** with every request, enabling multi-turn conversations. History is capped at 20 messages and 2,000 characters per message to control token usage.
- The backend endpoint is **stateless**: conversation state is owned entirely by the frontend React component.

8.4.2 Tool Use (Function Calling)

The chatbot is equipped with a **search_assets** tool that the AI model can invoke when a user asks about finding specific assets. When the model detects that the user wants to find something in the catalog, it calls the tool with a query string and optional category filter. The backend then executes the corresponding database query and returns the results for the model to incorporate into its response.

8.4.3 Frontend Widget

The `BotWidget.jsx` component renders as a fixed-position floating bubble in the bottom-right corner of every marketplace page. It opens a compact 360px-wide chat panel containing:

- A scrollable message history with distinct styling for user and assistant messages.
- A typing indicator (animated dots) while the AI is processing.
- Enter-to-send and Shift+Enter-for-newline keyboard shortcuts.
- A reset button to start a fresh conversation without page reload.
- Responsive design that adapts to mobile screen widths.

8.4.4 Remaining Work

The following items are planned for completion in Sprint 7:

1. **Tool call execution:** Complete the `search_assets` tool handler that queries PostgreSQL and feeds results back to the model for a grounded response.
2. **Streaming responses (SSE):** Replace the single-response endpoint with a streaming endpoint so users see the response being typed character-by-character.
3. **Tag suggestion:** Add an “AI Suggest Tags” action to the seller upload form.
4. **Response caching:** Cache common factual queries (pricing, formats, policies) to reduce API calls.

8.5 Future AI Perspectives

Beyond the current sprint scope, several AI enhancements are envisioned for future development:

- **Personalized Recommendations:** Use semantic embeddings and a vector database to recommend similar assets based on the currently viewed asset’s description, tags, and the user’s interaction history.
- **Automated Quality Assessment:** For uploaded 3D models, automatically assess polygon density, UV quality, and material complexity to produce a quality score at submission time.

- **AI-Assisted Moderation:** Pre-screen asset submissions for potential policy violations (inappropriate metadata, suspicious pricing patterns) before they enter the human moderation queue.
- **Dynamic Price Suggestions:** Suggest optimal pricing for new asset uploads based on comparable assets in the same category and their sales performance.

Conclusion

Sprint 6 has successfully integrated AI-powered review analysis and automated negative sentiment flagging into the IGS Marketplace. These features bring a new level of intelligence to the platform's seller tools and moderation workflow. The chatbot architecture is in place and its completion is the primary goal of Sprint 7, which also expands the recommendation engine and catalog taxonomy.

Chapter 9

Sprint 7: AI-Enhanced Services and Catalog Expansion

Introduction

Sprint 7 completes and extends the AI integration initiated in Sprint 6, delivers a fully operational marketplace assistant bot, expands the asset catalog to match industry-standard platforms with 6 new product categories (including Decals with full PBR workflow), rewrites the upload form to eliminate known bugs, enforces mandatory format selection before download, and populates every category with real seed assets hosted on Supabase CDN.

9.1 Sprint Backlog

Sprint 7 — April 2026

Goal: Deliver a complete AI recommendation engine, a floating marketplace chatbot, industry-standard product categories, and a fully operational catalog with real hosted assets.

Table 9.1: Sprint 7 — User Stories

ID	User Story	Priority	Status
US-45	As a user, I want AI-personalized asset recommendations on the home page based on my browsing, like, and purchase history.	High	Done
US-46	As a first-time visitor, I want to see trending assets when I have no history (cold start).	High	Done
US-47	As any user, I want a floating chatbot that can answer questions about the marketplace without leaving the page.	High	Done
US-48	As a buyer, I must select a file format before being allowed to download an asset.	Medium	Done
US-49	As a seller, I want the upload form to validate my input clearly and show errors per field.	Medium	Done
US-50	As a buyer, I want to browse Audio, VFX, 2D Assets, Animations, Environments, and Decals.	High	Done
US-51	As a seller, I want to upload Decals with PBR maps (Albedo, Normal, Roughness, Metallic, AO, Opacity).	Medium	Done
US-52	As a developer, I want one realistic example asset per category in the marketplace.	Low	Done

9.2 AI Recommendation Engine

9.2.1 Architecture — Two-Stage Pipeline

The recommendation system uses a **two-stage pipeline** combining fast content-based scoring with optional AI reranking:

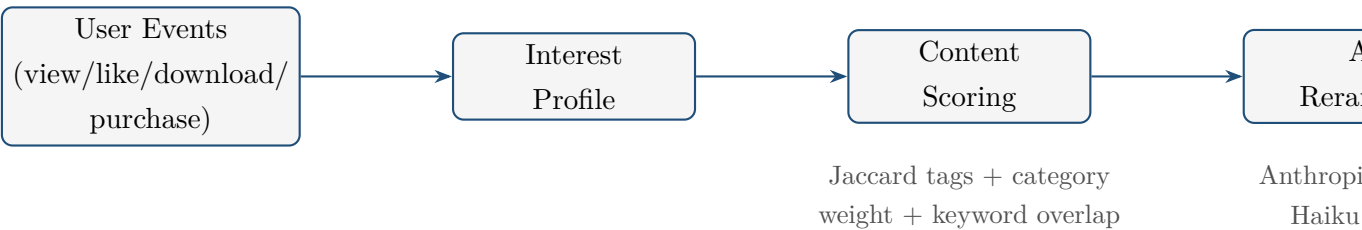


Figure 9.1: Two-Stage AI Recommendation Pipeline

1. **Stage 1 — Content-based scoring (no API call):** Every approved asset is

scored against the user’s interest profile using pure mathematical signals. This is fast, deterministic, and incurs no AI API cost.

2. **Stage 2 — AI reranking (optional):** The top-12 scored candidates are sent to the Anthropic AI API. The model reorders them based on the user’s profile context and returns a one-sentence explanation per recommended asset.

9.2.2 User Signal Tracking

A dedicated `user_events` table records every meaningful user interaction with an asset. Each event type is assigned a weight that reflects the strength of the user’s interest signal:

Table 9.2: User Signal Weights for Profile Building

Event Type	Weight
Purchase	10
Like	5
Download	3
View	1

Events are recorded in a fire-and-forget manner so they never block the primary API response. A `UNIQUE` constraint on `(user_id, asset_id, event_type)` ensures idempotent recording — viewing the same asset multiple times does not produce duplicate signals.

9.2.3 Content-Based Scoring

Each candidate asset is scored against the user’s interest profile using four signals:

- **Jaccard tag similarity** (0–5 pts): Measures overlap between the asset’s tags and the user’s top-10 weighted tags.
- **Category weight** (0–3 pts): Cumulative signal weight the user has accumulated in this asset’s category.
- **Keyword overlap** (0–2 pts): Word-level overlap between the asset’s title and the titles of previously interacted assets.
- **Popularity boost** (≤ 1.5 pts): A logarithmic function of the asset’s downloads, likes, rating, and rating count — prevents recommending obscure zero-interaction assets.

Assets the user has already interacted with are penalized (-1 score) and effectively excluded from the output.

9.2.4 AI Reranking

The top-12 scored candidates, the user’s interest profile summary, and a brief interaction history are sent to the Anthropic AI API (Haiku model, 1024 max tokens). The model returns a JSON array of asset IDs in its preferred order, along with a one-sentence explanation for each recommendation. If the API call fails or no key is configured, the system falls back gracefully to the content-based ranking.

9.2.5 Cold Start Handling

When a user has no interaction history:

1. The system returns trending assets from the last 30 days (ordered by signal score across all users).
2. If fewer than 5 trending assets exist, top-rated approved assets are returned instead.
3. Unauthenticated visitors always receive top-rated assets.

The home page displays either “*Recommended for You*” (personalized) or “*Trending Now*” (cold start) based on the `coldStart` flag returned by the API.

9.3 IGS Marketplace Bot

9.3.1 Overview

The IGS Bot is a **floating chat widget** available on every page of the marketplace. It is powered by the Anthropic AI API (Haiku model) with a marketplace-specific system prompt, and it can answer questions about assets, categories, the seller programme, trading, subscriptions, and file formats.

9.3.2 AI Model Configuration

Table 9.3: IGS Bot — AI Model Configuration

Parameter	Value
Model	Anthropic Haiku (<code>claude-haiku-4-5-20251001</code>)
Max tokens per reply	512
Conversation cap	20 messages, 2,000 characters each
System prompt scope	Full marketplace knowledge + user role context + current page
Tool use	<code>search_assets</code> (catalog search by keyword/category)
State management	Stateless backend; conversation history owned by frontend

9.3.3 Context Injection

The system prompt injects three layers of context into every conversation:

1. **Platform knowledge:** All 13 asset categories, supported file formats, pricing model, seller programme requirements, trading rules, subscription tiers, and platform policies.
2. **User context:** The authenticated user’s role (guest / member / seller / internal staff), enabling role-appropriate responses (e.g., explaining seller upload steps only to sellers).
3. **Page context:** The current route path sent by the frontend, allowing the bot to give page-specific guidance.

9.3.4 Frontend Widget Features

The `BotWidget.jsx` floating widget provides:

- A persistent floating bubble (bottom-right corner) that opens a 360 px chat panel.
- Conversation history held in React state and never persisted between page sessions.
- Enter-to-send and Shift+Enter-for-newline keyboard shortcuts.
- Animated typing indicator while the AI is processing.

- Markdown-lite rendering (bold text, line breaks) from the AI’s plain-text responses.
- Reset button to clear the session without page reload.
- Responsive mobile adaptation.

9.4 Upload Form Improvements

The original upload form stored all fields in a single shared state object, which caused a stale closure bug in React’s asynchronous state batching — file change handlers captured old snapshots of the form data, silently discarding user input. Additionally, a global CSS rule targeting all `<input>` elements was crushing checkboxes.

The improvements delivered in Sprint 7:

Table 9.4: Upload Form — Key Improvements

Area	Before	After
State management	Single shared state object	One <code>useState</code> hook per field
File handlers	Stale closure, silent data loss	Direct setter, always current value
Checkboxes	Not clickable (CSS override)	Fixed with scoped CSS selector
Validation	Server-side only	Client-side per-field with inline errors
Post-upload flow	Redirect only	“Upload another” resets form cleanly

9.5 Mandatory Format Selection Before Download

Assets with multiple download formats (e.g., GLB + OBJ + FBX) now require the buyer to explicitly select a format before the download button becomes active. This eliminates accidental downloads and ensures buyers receive the format suited to their pipeline:

- The format selector highlights the selected format; deselection is disabled once a choice is made.
- The download button remains disabled while no format is selected, with a helper indicator.

- A toast warning appears if the button is clicked without a selection via keyboard shortcuts.

9.6 Catalog Expansion — New Asset Categories

Inspired by **Fab.com** (Epic Games’ unified asset marketplace), six new product categories were added to match industry-standard taxonomy:

Table 9.5: New Product Categories Added in Sprint 7

Category	Slug	Description	Accepted Formats
Animations	animations	Character animations, motion capture, BVH rigs	.fbx, .glb, .bvh, .zip
Environments	environments	Full scenes, levels, landscape kits	.fbx, .glb, .zip
Audio	audio	SFX, music loops, ambient tracks	.mp3, .wav, .ogg, .flac
VFX	vfx	Particles, explosions, magic effects	.zip, .unitypackage
2D Assets	2d	Sprites, UI kits, concept art	.png, .psd, .svg, .zip
Decals	decals	PBR surface overlays: damage, dirt, graffiti	.png, .jpg (maps)

The sidebar navigation was reorganized into four labelled sections mirroring Fab’s channel structure:

- **Discover** — Home, Trending
- **3D & Models** — Models, Animations, Environments, Printable
- **Materials & FX** — Textures, HDRIs, Shaders, VFX, Decals
- **Tools & Other** — Scripts, Plugins, Audio, 2D Assets

The Decals category includes a dedicated upload form with six PBR map inputs: Albedo (required), Normal (required), Roughness, Metallic, Ambient Occlusion, and Opacity/Alpha. A **Decal Type** selector (damage, dirt, graffiti, weathering, effects, other) and a **Surface Type** selector (concrete, wood, metal, asphalt, universal) provide professional categorization aligned with industry workflow standards.

9.7 Seed Data — Real Assets on Supabase CDN

To demonstrate every category at launch, 13 example assets were prepared using publicly licensed (CC0 / MIT) files downloaded from Polyhaven, KhronosGroup, and other open repositories, then uploaded to Supabase Storage:

Table 9.6: Sprint 7 Seed Assets

Category	Asset Title	Source	Size
Models	Sci-Fi Battle Drone	KhronosGroup (Avocado.glb)	7.7 MB
Animations	Warrior Combat Animation Set	KhronosGroup (Fox.glb)	0.2 MB
Environments	Desert Ruins — Full Scene	KhronosGroup (BoomBox.glb)	10.1 MB
Textures	Mossy Cobblestone PBR Pack	Polyhaven (cobblestone_floor_08)	0.6 MB
HDRIs	Sunset Grasslands HDRI (4K)	Polyhaven (studio_small_08_1k)	1.4 MB
Shaders	Holographic Shield Shader	libretro/glsl-shaders	0.01 MB
Scripts	Auto LOD Generator (Blender)	blender/blender-addons	0.01 MB
Plugins	Smart Texture Importer	jayanam/fast-carve	0.02 MB
Printables	Medieval Castle Tower (STL)	three.js examples	0.2 MB
Audio	Sci-Fi Ambience Pack	rafaelreis/Audio-Sample- files	0.9 MB
VFX	Magic Spell Particle System	gdquest/godot-4-VFX- assets	1.9 MB
2D Assets	Fantasy UI Kit (Vector)	three.js examples (sprite.png)	0.02 MB
Decals	Concrete Damage Decal Pack	Polyhaven (concrete_wall_001)	0.9 MB

All seed assets are set to `status = 'approved'` and are immediately visible in the public catalog. The seeder scripts are idempotent and safe to re-run without creating duplicates.

9.8 Database Changes

Table 9.7: Sprint 7 — Database Migrations

Migration File	Description
016_user_events.sql	<code>user_events</code> table for recommendation signal tracking with UNIQUE constraint on (<code>user_id</code> , <code>asset_id</code> , <code>event_type</code>).
017_new_categories.sql	Inserts 5 new categories: Audio, VFX, 2D Assets, Animations, Environments.
018_decals_category.sql	Inserts the Decals category (slug: <code>decals</code>).

Conclusion

Sprint 7 delivered the platform’s most user-facing AI features and significantly expanded the asset catalog. The recommendation engine brings personalized discovery to authenticated users while providing a polished trending experience to new visitors. The marketplace bot removes friction by answering questions in context without leaving the current page. The addition of 6 industry-inspired categories raises the platform’s scope to match international marketplace standards, and the real seed data gives every category a tangible starting point for demonstration.

General Conclusion and Perspectives

Summary of Achievements

This end-of-studies project has resulted in the successful design and development of **IGS Marketplace**, a full-featured digital asset marketplace platform built for Inherited Games Studio. Over the course of six Scrum sprints, the following was delivered:

- **Sprint 1 — Core Infrastructure:** A robust Express.js + PostgreSQL backend with JWT + OAuth2 authentication and a fine-grained six-level RBAC system.
- **Sprint 2 — Asset Marketplace Core:** A fully functional e-commerce experience with an interactive 3D/texture/HDRI viewer, Supabase file storage, a persistent shopping cart, and Stripe payment integration.
- **Sprint 3 — Content Quality & Administration:** A review and rating system, a real-time notification center, a three-tier subscription model, and a comprehensive admin moderation panel with audit logging.
- **Sprint 4 — Advanced Role Features:** An intern asset request and voucher workflow, seller analytics, asset edit requests, peer-to-peer trading, and community groups.
- **Sprint 5 — IGS Workspace:** A companion internal web application with team management, task tracking, project ERP, and a real-time chat system powered by Socket.io — sharing authentication seamlessly with the marketplace.
- **Sprint 6 — AI Integration (In Progress):** Integration of the Anthropic Claude API for automated review sentiment analysis and the in-progress implementation of a marketplace assistant chatbot with tool-use capabilities.

The platform successfully addresses all original pain points: it provides a centralized, searchable asset repository; a monetization mechanism for creators; strict RBAC with six distinct permission levels; a structured intern management workflow; and a unified team collaboration tool — all within a single cohesive ecosystem.

Technical Lessons Learned

Throughout this project, several important technical insights were gained:

- **RBAC design is foundational:** Designing the permission system in Sprint 1 and respecting it consistently through all subsequent sprints was critical. Retrofitting access control is significantly more costly than building it correctly from the start.
- **Cloud storage abstraction pays off:** Wrapping Supabase Storage in a `storageService` abstraction made it trivial to change storage parameters without touching business logic.
- **PostgreSQL JSONB for flexible metadata:** Using a JSONB `metadata` column on the `assets` table allowed each asset category to store category-specific data (vertex counts, PBR map types, etc.) without requiring separate tables — a pragmatic balance between flexibility and structure.
- **Scrum for solo development:** Applying Scrum as a solo developer required discipline and adaptation. The sprint structure proved effective for managing scope and avoiding scope creep, even without a full team.
- **AI integration complexity:** Integrating an AI model is straightforward for simple use cases (sentiment analysis) but requires careful design for interactive features (chatbots with tool use), particularly around state management, streaming, and error handling.

Perspectives and Future Work

The IGS Marketplace platform is designed for extensibility. Several directions are planned for future development:

1. **Complete AI Integration:** Finalize the chatbot with full tool-use capability, add streaming responses, and implement AI-based tag suggestions and smart recommendations.
2. **Mobile Application:** Develop a React Native mobile client for the marketplace, allowing buyers to browse and purchase assets on mobile devices.
3. **Advanced Analytics:** Expand the analytics dashboard with time-series charts, cohort analysis, and seller revenue predictions.
4. **Automated Testing Suite:** Add comprehensive unit and integration tests (Jest, Supertest) to ensure code quality as the platform scales.
5. **Production Deployment:** Deploy the platform on a cloud provider (e.g., Railway, Render, or AWS) with a proper CI/CD pipeline, environment management, and HTTPS.

6. **Licensing System:** Add a digital asset licensing framework where sellers can define the usage rights (personal, commercial, extended) associated with their assets.
7. **Community Features Expansion:** Add asset bundle creation (sellers package multiple assets), wishlists, and a social following system between creators and buyers.

Closing Remarks

This project has been an extraordinarily enriching experience — both technically and professionally. It provided the opportunity to work across the full stack of modern web development, from database design and REST API architecture to React component design, real-time systems, payment processing, and AI integration. The Scrum methodology kept the work organized and deliverable-focused despite the broad scope.

Most importantly, this project addresses a real business need for Inherited Games Studio and delivers a production-ready foundation that the studio can continue to build upon. It represents not just the conclusion of a degree, but the beginning of a platform that can grow with the studio's ambitions.

Bibliography

- [1] OpenJS Foundation. *Node.js Documentation*. 2024. <https://nodejs.org/en/docs/>
- [2] StrongLoop. *Express.js Guide*. 2024. <https://expressjs.com/en/guide/routing.html>
- [3] Meta Platforms. *React Documentation*. 2024. <https://react.dev/>
- [4] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. 2024. <https://www.postgresql.org/docs/16/>
- [5] Supabase Inc. *Supabase Documentation — Storage*. 2024. <https://supabase.com/docs/guides/storage>
- [6] Stripe, Inc. *Stripe API Reference*. 2024. <https://stripe.com/docs/api>
- [7] Google LLC. *Firebase Authentication Documentation*. 2024. <https://firebase.google.com/docs/auth>
- [8] Socket.IO contributors. *Socket.IO Documentation*. 2024. <https://socket.io/docs/v4/>
- [9] Mr.doob et al. *Three.js Documentation*. 2024. <https://threejs.org/docs/>
- [10] Anthropic. *Claude API Documentation*. 2024. <https://docs.anthropic.com/>
- [11] Schwaber, K. & Sutherland, J. *The Scrum Guide*. November 2020. <https://scrumguides.org/scrum-guide.html>
- [12] Object Management Group. *UML 2.5.1 Specification*. 2017. <https://www.omg.org/spec/UML/2.5.1>
- [13] Jones, M. et al. *RFC 7519 — JSON Web Token (JWT)*. IETF, 2015. <https://datatracker.ietf.org/doc/html/rfc7519>
- [14] Ferraiolo, D. & Kuhn, R. *Role-Based Access Control*. NIST, 2009.
- [15] P Market Research. *Worldwide 3D Marketplace Models Market Research 2024 — by Type, Application, Participants and Countries, Forecast to 2030*. 2024. <https://pmarketresearch.com/worldwide-3d-marketplace-models-market-research-2024-by-type-application-parti>

- [16] Citius Research. *3D Project Market Report*. 2024. <https://citiusresearch.com/publication-details/3d-project-market-report>

Abstract

ملخص

يُقدّم هذا المشروع تصميم وتطوير منصة **Marketplace IGS**، وهي منصة ويب متكاملة لبيع وشراء الأصول الرقمية، أنجزت لصالح شركة **(IGS) Studio Games Inherited**، وهي شركة تونسية متخصصة في تطوير ألعاب الفيديو، وتُعدّ أول منصة مخصصة للأصول الرقمية ثلاثية الأبعاد تُصمّم وتُطوّر في تونس. تُتيح المنصة للمبدعين نشر أصولهم الرقمية وإدارتها وتحقيق الدخل منها (نماذج 3D، مواد رسومية، نصوص برمجية، ملفات طباعة)، مع وصول مُنظّم وفق ستة مستويات للصلاحيات. يعتمد النظام على **Vite + React** للواجهة، و**Express.js + PostgreSQL** لل خادم، مع **Supabase** للتخزين و**Firebase** للمصادقة. وتشمل الميزات: عارضاً تفاعلياً 3D، ودفع آمن عبر **Stripe** ونظام تقييم، وإشعارات، وتبادل أصول، وتطبيقاً داخلياً **Workspace IGS** للتعاون الفوري، فضلاً عن طبقة ذكاء اصطناعي للتوصيات والمساعد الحواري. طُوّر المشروع وفق منهجية **Scrum** في سبعة سبرينتات.

الكلمات المفتاحية: منصة أصول رقمية، **RBAC**، **React**، **Node.js**، **PostgreSQL**، **Stripe** ذكاء اصطناعي.

English

This project presents the design and development of **IGS Marketplace**, a full-stack digital asset marketplace platform built for Inherited Games Studio (IGS), a Tunisian video game company — and the **first dedicated 3D digital asset marketplace designed and built in Tunisia**. The platform enables creators to publish, manage, and monetize digital assets (3D models, textures, shaders, scripts, HDRIs, plugins, printables) with structured, role-governed access across six roles. Built on **React + Vite / Express.js + PostgreSQL**, with Supabase storage and Firebase authentication, key features include: an interactive 3D viewer, Stripe payment integration, reviews and ratings, a real-time notification center, peer-to-peer trading, subscription plans, community groups, an intern access workflow, and the companion **IGS Workspace** for team collaboration and real-time chat. An AI layer provides intelligent review analysis, personalized recommendations, and a conversational chatbot. Developed with **Scrum** over seven sprints.

Keywords: Digital Asset Marketplace, Full-Stack, RBAC, Scrum, React, Node.js, PostgreSQL, 3D Viewer, Stripe, Artificial Intelligence.

Résumé (Français)

Ce projet présente la conception et le développement de **IGS Marketplace**, une plateforme full-stack de marketplace d'assets numériques réalisée pour Inherited Games Studio (IGS), société tunisienne de développement de jeux vidéo — et **la première marketplace dédiée aux assets 3D conçue et développée en Tunisie**. La plateforme permet aux créateurs de publier, gérer et monétiser des assets numériques (modèles 3D, textures, shaders, scripts, HDRIs, plugins, fichiers imprimables) avec un accès gouverné par six niveaux de rôles. Bâtie sur **React**

+ **Vite** / **Express.js** + **PostgreSQL**, avec Supabase et Firebase, les fonctionnalités incluent : visualiseur 3D interactif, paiement Stripe, avis et notes, notifications temps réel, échange pair-à-pair, abonnements, groupes communautaires, flux de demandes stagiaires, et l'application interne **IGS Workspace** pour la collaboration et le chat. Une couche IA assure l'analyse des avis, les recommandations personnalisées et un chatbot. Développé avec **Scrum** en sept sprints.

Mots-clés : Marketplace d'assets numériques, Full-Stack, RBAC, Scrum, React, Node.js, PostgreSQL, Stripe, Intelligence Artificielle.
